PRAXISHANDBUCH

Vibe Coding unter Linux

Das Praxishandbuch für Einsteiger mit Linux-Grundkenntnissen



KLAR BESCHREIBEN. KLEIN BAUEN. SICHER PRÜFEN.

Mit Übungen, Checklisten, Glossar und Index · Ausgabe Oktober 2026

Vibe Coding unter Linux

Das Praxishandbuch für Einsteiger mit Linux-Grundkenntnissen

Von der Idee zum funktionierenden Programm – mit KI-Assistenten, Terminal und Git.
Mit drei Praxisprojekten, Übungen, Glossar und Index.

Inhaltsverzeichnis

Vorwort 5

Einleitung: So arbeitest du mit diesem Handbuch 6

Für wen ist dieses Buch? 6

Aufbau 6

Jedes Kapitel funktioniert gleich 6

Konventionen für Code und Terminal 7

Kapitel 1: Was ist Vibe Coding? 8

1.1 Definition und Ursprung 8

1.2 Klassisches Programmieren und Vibe Coding im Vergleich 8

1.3 Was ein KI-Assistent eigentlich tut 9

1.4 Was geht – und was nicht 10

1.5 Dein Mindset: Auftraggeber, Tester, Lerner 10

Übungen 11

Zusammenfassung 11

Kapitel 2: Setup der Umgebung unter Linux 13

2.1 Die Grundausstattung 13

2.2 Der Arbeitsordner 14

2.3 Python und virtuelle Umgebungen 15

2.4 Der Editor 16

2.5 KI-Assistenten einrichten 17

2.6 API-Keys – was sie sind und wie du sie ablegst 18

2.7 Git – dein Sicherheitsnetz 18

2.8 Projektvorlage zum Wiederverwenden 20

Übungen 21

Zusammenfassung 21

Kapitel 3: Der Vibe-Coding-Workflow 23

3.1 Der Grundkreislauf 23

3.2 Vor dem ersten Prompt: Plan und Projektnotiz 23

3.3 Code erzeugen, lesen, ausführen 24

3.4 Iterieren: Änderungswünsche formulieren 26

3.5 Debugging mit KI 27

3.6 Kontext-Management 28

3.7 Wenn nichts mehr geht: Notausgänge . . . 29
3.8 Das Lerntagebuch . . . 29
Übungen . . . 29
Zusammenfassung . . . 30

Kapitel 4: Prompting-Grundlagen für Code . . . 31

4.1 Was einen guten Prompt ausmacht . . . 31
4.2 Bewährte Muster . . . 32
4.3 Kontext liefern: Was, wie viel, in welcher Form . . . 32
4.4 Fehlermeldungen richtig einspeisen . . . 33
4.5 Verständnisfragen – so lernst du nebenbei . . . 34
4.6 Code-Reviews durch KI . . . 34
4.7 Tests als Prompt-Hilfe . . . 35
Übungen . . . 35
Zusammenfassung . . . 36

Kapitel 5: Drei Praxisprojekte . . . 37

Projekt 1: dupfinder – ein CLI-Tool in Python . . . 37
Projekt 2: Ein kleines Web-Projekt . . . 42
Projekt 3: Ein Automatisierungsskript . . . 46
Übungen . . . 51
Zusammenfassung . . . 51

Kapitel 6: Sicherheit und Qualität . . . 52

6.1 Warum Sicherheit hier besonders wichtig ist . . . 52
6.2 API-Keys und Geheimnisse . . . 52
6.3 Gefährliche Befehle erkennen . . . 53
6.4 Typische Schwachstellen in KI-Code . . . 54
6.5 Qualitätssicherung in der Praxis . . . 56
6.6 Abhängigkeiten und Lieferkette . . . 56
6.7 Lizenzfragen . . . 57
6.8 Datenschutz . . . 57
6.9 Wann du KI-Code NICHT vertrauen darfst . . . 58
Übungen . . . 59
Zusammenfassung . . . 59

Kapitel 7: Typische Fallstricke und Troubleshooting . . . 60

7.1 Halluzinationen . . . 60
7.2 Veralteter Code und Versionskonflikte . . . 61

7.3 Abhängigkeitskonflikte . . . 61
7.4 „Es funktioniert, aber ich verstehe es nicht“ . . . 62
7.5 Weitere Fallstricke . . . 63
7.6 Ein Fahrplan für jedes Problem . . . 64
Übungen . . . 64
Zusammenfassung . . . 65

Kapitel 8: Nächste Schritte . . . 66

8.1 Wo stehst du jetzt? . . . 66
8.2 Der Lernpfad in vier Stufen . . . 66
8.3 Die KI als Lehrer einsetzen . . . 67
8.4 Folgeprojekte nach Interesse . . . 67
8.5 Ressourcen zum Weiterlernen . . . 68
8.6 Berufliche Perspektiven realistisch einschätzen . . . 69
8.7 Deine Alltags-Checkliste . . . 69
Schlusswort . . . 70
Übungen . . . 70
Zusammenfassung . . . 70

Glossar . . . 72

Verzeichnis der Codebeispiele . . . 74

Index . . . 76

Vorwort

Du kennst das Terminal. Du weißt, was `cd`, `ls` und `grep` tun, du hast schon einmal ein Paket mit `apt` oder `dnf` installiert und dich über eine Fehlermeldung geärgert. Was dir bisher fehlt, ist der Schritt vom **Benutzer** zum **Macher**: Du möchtest nicht nur Programme verwenden, sondern eigene bauen – ein kleines Werkzeug, das dir jeden Tag Arbeit abnimmt, eine Webseite, ein Skript, das Backups sortiert.

Früher bedeutete das: monatelang eine Programmiersprache lernen. Heute gibt es einen zweiten Weg. KI-Assistenten schreiben Code, erklären ihn, suchen Fehler und schlagen Verbesserungen vor. Dieser Weg hat einen Namen bekommen: **Vibe Coding**. Du beschreibst, was du möchtest; die KI liefert einen Entwurf; du testest, korrigierst die Richtung und wiederholst das, bis das Ergebnis passt.

Dieses Handbuch ist ehrlich. Es verspricht dir nicht, dass du ohne jedes Verständnis professionelle Software bauen kannst. Es zeigt dir aber, wie du mit Linux-Grundwissen, einem klaren Kopf und einem guten Arbeitsablauf erstaunlich weit kommst – und wie du dabei sicher und sauber arbeitest. Du lernst nebenbei die wichtigsten Grundlagen des Programmierens, nicht als trockene Theorie, sondern genau dann, wenn du sie brauchst.

> **Gut zu wissen**
>
> **Stand der Inhalte:** Dieses Handbuch ist im Oktober 2026 entstanden. Werkzeuge, Preise, Modellnamen und Menüs ändern sich im KI-Bereich schnell. Die **Prinzipien** (Kapitel 3, 4, 6, 7) bleiben stabil; für Installationsschritte und Funktionen prüfst du bitte immer die offizielle Dokumentation des jeweiligen Werkzeugs. Wo das wichtig ist, steht ein Hinweis im Text.

Einleitung: So arbeitest du mit diesem Handbuch

Für wen ist dieses Buch?

Du bist hier richtig, wenn du:

- im Terminal sicher genug bist, um Dateien anzulegen, zu verschieben, zu suchen und Rechte zu setzen,
- bisher wenig oder gar nicht programmiert hast,
- mit KI-Assistenten (zum Beispiel Claude, ChatGPT oder GitHub Copilot) eigene kleine Software bauen möchtest.

Du brauchst **kein** Vorwissen in Python, JavaScript oder Git. Das Wichtigste dazu lernst du in den Kapiteln.

Aufbau

Das Handbuch hat acht Kapitel, die aufeinander aufbauen:

Kapitel	Thema	Du kannst danach
1	Was ist Vibe Coding?	Erwartungen und Grenzen realistisch einschätzen
2	Setup unter Linux	Editor, KI-Assistent, Git und Projektordner einrichten
3	Der Workflow	in kleinen Schritten vom Prompt zum laufenden Code kommen
4	Prompting für Code	präzise Anweisungen und Fehlermeldungen formulieren
5	Drei Praxisprojekte	CLI-Tool, Web-Projekt und Automatisierungsskript bauen
6	Sicherheit und Qualität	Keys, Daten und Code verantwortungsvoll behandeln
7	Fallstricke	typische Probleme erkennen und beheben
8	Nächste Schritte	deinen Lernpfad planen

Am Ende findest du ein **Glossar**, ein **Verzeichnis der Codebeispiele** und einen **Index**.

Jedes Kapitel funktioniert gleich

Jedes Kapitel beginnt mit **Lernzielen** und endet mit **Übungen** und einer **Zusammenfassung**. Dazwischen findest du Erklärungen und Codebeispiele. Die Beispiele sind nummeriert (zum Beispiel „Beispiel 3.2“), damit du sie im Verzeichnis am Ende wiederfindest.

Es gibt farbige Boxen:

WARNUNG – Sicherheitsrisiko

Rote Boxen warnen vor echten Risiken: Datenverlust, ausgespähte Zugangsdaten, gefährliche Befehle. Lies sie immer.

TIPP – Profi-Trick

Grüne Boxen enthalten Tricks, die den Alltag deutlich leichter machen.

Gut zu wissen

Violette Boxen geben Hintergrundwissen, das du nicht sofort brauchst, das aber Zusammenhänge klärt.

Konventionen für Code und Terminal

Codeblöcke mit einem `$` am Zeilenanfang sind Terminal-Befehle. Das `$` tippst du **nicht** mit; es zeigt nur die Eingabeaufforderung. Zeilen ohne `$` darunter sind die Ausgabe. Ein `#` in einer Shell-Zeile leitet einen Kommentar ein.

Die Beispiele wurden für gängige Distributionen (Debian, Ubuntu, Fedora, Arch) geschrieben. Wenn sich Befehle unterscheiden, stehen die Varianten nebeneinander. Als Arbeitsordner verwenden wir durchgehend `~/vibe` (also `/home/DEINNAME/vibe`).

TIPP – Profi-Trick

Arbeite jedes Beispiel wirklich am Rechner durch. Beim Programmieren lernst du fast nur durch Tun. Wer nur liest, versteht später zwar die Wörter, aber nicht die Probleme.

Kapitel 1: Was ist Vibe Coding?

Lernziele

- Du kannst Vibe Coding definieren und vom klassischen Programmieren abgrenzen.
- Du kennst den Ursprung des Begriffs und die Bandbreite seiner Bedeutung.
- Du verstehst, was ein KI-Assistent technisch tut – und warum er Fehler macht.
- Du kannst realistisch einschätzen, was mit Vibe Coding geht und was nicht.

1.1 Definition und Ursprung

Der Begriff **Vibe Coding** wurde im Februar 2025 durch den KI-Forscher Andrej Karpathy populär. In einem Beitrag beschrieb er eine Arbeitsweise, bei der man sich „den Vibes hingibt“: Man sagt der KI in normaler Sprache, was man möchte, übernimmt ihre Vorschläge, ohne jede Zeile im Detail zu prüfen, und korrigiert nur, wenn etwas nicht funktioniert. Der Begriff verbreitete sich rasend schnell und wurde 2025 sogar von einem großen Wörterbuchverlag zum Wort des Jahres gekürt.

Karpathys ursprüngliche Beschreibung meinte eine bewusst lockere Arbeitsweise für Wochenendprojekte und Experimente. Im Alltag hat der Begriff sich verbreitert. Heute fallen darunter drei verschiedene Dinge:

Variante	Beschreibung	Typischer Einsatz
Reines Vibe Coding	Code wird nicht gelesen, nur das Ergebnis getestet	Wegwerf-Prototypen, Spielereien
KI-gestütztes Programmieren	Die KI schreibt, der Mensch liest und versteht den Code	professionelle Entwicklung
Geführtes Lernen mit KI	Die KI baut und erklärt, der Mensch lernt dabei	genau dein Weg mit diesem Buch

In diesem Handbuch gehen wir bewusst den **mittleren bis dritten Weg**: Du lässt dir Code schreiben, aber du lernst, ihn zu testen, in groben Zügen zu lesen und zu hinterfragen. Das ist langsamer als reines Vibe Coding, aber sehr viel robuster.

Gut zu wissen

Faustregel: Je mehr Schaden ein Fehler anrichten kann, desto weniger darfst du „nach Gefühl“ arbeiten. Ein Skript, das Fotos umbenennt, darf mal schiefgehen (mit Backup). Ein Skript, das Passwörter verwaltet oder Geld bewegt, darfst du nie ungeprüft einsetzen.

1.2 Klassisches Programmieren und Vibe Coding im Vergleich

Beim klassischen Programmieren schreibst du jede Anweisung selbst. Du musst die Syntax einer Sprache beherrschen, die Standardbibliothek kennen und Fehler selbst suchen. Dafür weißt du genau, was dein Programm tut.

Beim Vibe Coding verschiebt sich deine Rolle: Du wirst vom Schreibenden zum **Auftraggeber, Tester und Prüfer**. Die Fähigkeiten, die du brauchst, ändern sich:

Fähigkeit	Klassisch	Vibe Coding
Syntax auswendig	sehr wichtig	kaum nötig
Problem klar beschreiben	wichtig	**entscheidend**
Ergebnis testen	wichtig	**entscheidend**
Fehlermeldungen lesen	wichtig	wichtig
Aufgabe in kleine Schritte teilen	wichtig	**entscheidend**
Code im Detail verstehen	Pflicht	wünschenswert, wächst mit der Zeit
Versionskontrolle (Git)	wichtig	**sehr wichtig** (Sicherheitsnetz)

Beachte: Die letzte Zeile ist kein Zufall. Wenn eine KI in Sekunden hunderte Zeilen ändert, brauchst du ein Sicherheitsnetz, mit dem du jederzeit zum letzten funktionierenden Stand zurückkehren kannst. Dieses Netz heißt Git, und du richtest es in Kapitel 2 ein.

1.3 Was ein KI-Assistent eigentlich tut

Ein **Sprachmodell** (englisch: Large Language Model, LLM) wurde mit riesigen Mengen Text und Code trainiert. Es hat dabei Muster gelernt: Wie sieht eine Python-Funktion aus, die eine Datei liest? Wie sieht eine typische Fehlermeldung aus, und was ist meist die Ursache?

Wenn du eine Anfrage stellst, **berechnet** das Modell Wort für Wort (genauer: Token für Token), was wahrscheinlich als Nächstes kommt. Es „führt dabei keinen Code aus“ und „schlägt nichts nach“, solange es nicht dafür ausgerüstet wurde. Daraus folgen drei wichtige Eigenschaften:

- **Es klingt immer selbstsicher.** Auch falsche Antworten sind flüssig formuliert.
- **Es kann Dinge erfinden** (Halluzinationen): Funktionen, die es nicht gibt, Kommandozeilenoptionen, die nie existiert haben, Bibliotheken mit erfundenem Namen.
- **Sein Wissen hat ein Datum.** Neuere Versionen von Bibliotheken kennt es möglicherweise nicht.

Moderne Assistenten haben diese Schwächen durch Werkzeuge abgemildert: Manche können Dateien in deinem Projekt lesen, Befehle im Terminal ausführen, Tests laufen lassen und im Web nachschlagen. Das macht sie deutlich nützlicher, aber nicht unfehlbar.

Drei Arten von Assistenten

1. **Chat-Assistenten im Browser** (zum Beispiel Claude, ChatGPT, Gemini): Du kopierst Code hin und her. Ideal zum Einstieg und für Erklärungen.
2. **Editor-Assistenten** (zum Beispiel GitHub Copilot, Continue oder KI-Editoren wie Cursor): Die KI sitzt im Editor, sieht deine geöffneten Dateien und schlägt Änderungen direkt im Code vor.
3. **Terminal-Agenten** (zum Beispiel Claude Code, Aider, Codex CLI oder Gemini CLI): Die KI läuft in deinem Terminal, liest dein Projektverzeichnis, ändert Dateien und führt Befehle aus – mit deiner Erlaubnis.

> **Gut zu wissen**
>
> Für Linux-Nutzer sind Terminal-Agenten besonders spannend, weil sie genau in der Umgebung arbeiten, die du ohnehin kennst. Sie sind aber auch die Werkzeuge mit dem größten Schadenspotenzial, weil sie Befehle ausführen. Mehr dazu in Kapitel 6.

1.4 Was geht – und was nicht

Das gelingt in der Regel gut

- **Kleine bis mittlere Werkzeuge:** Skripte für Dateien, Texte, Backups, Umbenennungen, Auswertungen.
- **Prototypen und Webseiten:** Eine einfache Seite, ein Formular, ein kleiner Rechner, ein Dashboard.
- **Glue-Code:** Ein Programm mit einem anderen verbinden, Daten von A nach B bringen.
- **Gängige Muster:** Alles, was tausendfach im Netz vorkommt (Login-Formular, CSV einlesen, Tabelle sortieren).
- **Lernen und Erklären:** „Erkläre mir diese Zeile“ ist eine der besten Anwendungen überhaupt.

Hier wird es schwierig

- **Große, gewachsene Systeme**, in denen die KI den Gesamtzusammenhang nicht überblickt.
- **Sicherheitskritische Bereiche:** Authentifizierung, Kryptografie, Zahlungsverkehr.
- **Sehr neue oder seltene Technik**, über die es wenig Material gibt.
- **Unklare Anforderungen:** Wenn du selbst nicht weißt, was du möchtest, kann die KI es nicht erraten.
- **Performance und Feinheiten:** Code, der „läuft“, ist nicht automatisch schnell oder wartbar.

Die ehrliche Erwartungskurve

Viele Einsteiger erleben denselben Verlauf: Die ersten Erfolge sind verblüffend – nach zehn Minuten läuft ein Programm. Dann wächst das Projekt, die KI ändert Dinge, die sie nicht ändern sollte, und plötzlich geht nichts mehr. Das ist normal. Der Rest dieses Buches zeigt dir, wie du diese zweite Phase überstehst: mit kleinen Schritten, Git, Tests und gezielten Fragen.

> **WARNUNG – Sicherheitsrisiko**
>
> **Gefährliche Illusion:** „Es läuft, also ist es richtig.“ Code kann ohne Fehlermeldung laufen und trotzdem das Falsche tun – zum Beispiel Dateien überschreiben, Daten verlieren oder Lücken öffnen. Teste immer an Kopien und nie zuerst an deinen echten Daten.

1.5 Dein Mindset: Auftraggeber, Tester, Lerner

Drei Rollen begleiten dich durch das ganze Handbuch:

1. **Auftraggeber:** Du formulierst klar, was gebaut werden soll und was nicht.
2. **Tester:** Du führst das Ergebnis aus, probierst Randfälle und prüfst, ob es dein Ziel erfüllt.
3. **Lerner:** Du fragst bei jedem unverständlichen Teil nach. Aus „Es funktioniert, aber ich verstehe es nicht“ wird mit der Zeit „Ich verstehe die Hauptidee“.

Ein sinnvolles Ziel ist nicht, jede Zeile zu verstehen, sondern **den Aufbau**: Welche Teile gibt es? Welcher Teil tut was? Wo würde ich suchen, wenn etwas kaputt ist?

Dein erster Mini-Test: Hallo Welt mit einer KI

Bevor wir die Umgebung einrichten, ein kurzer Vorgeschmack. Öffne einen Chat-Assistenten im Browser und gib diese Anfrage ein:

Beispiel 1.1: Eine erste Anfrage an die KI

```
Schreibe mir ein kleines Python-Programm für Linux. Es soll mich nach meinem
Namen fragen und dann "Hallo, <Name>!" ausgeben. Erkläre danach jede Zeile
in einem Satz, als wäre ich Anfänger.
```

Du erhältst etwa ein Programm wie dieses:

Beispiel 1.2: Typische Antwort – ein Hallo-Programm

```
name = input("Wie heißt du? ")
print(f"Hallo, {name}!")
```

Speichere es als `~/hallo.py` und starte es im Terminal:

Beispiel 1.3: Programm starten

```
$ python3 ~/hallo.py
Wie heißt du? Eugen
Hallo, Eugen!
```

Was ist passiert? `input(...)` zeigt eine Frage an und wartet auf deine Eingabe. Das Ergebnis speichert das Programm in der **Variablen** `name`. `print(...)` gibt Text aus; das `f` vor den Anführungszeichen erlaubt, mit `{name}` den Inhalt der Variable in den Text einzusetzen. Mehr ist das erste Programm nicht – und du hast es gerade durch eine Beschreibung in normaler Sprache entstehen lassen.

Übungen

Übung

1. Formuliere in eigenen Worten den Unterschied zwischen reinem Vibe Coding und KI-gestütztem Programmieren.
2. Nenne je ein Beispielprojekt, bei dem „nach Gefühl“ arbeiten okay ist, und eines, bei dem es riskant wäre.
3. Lass dir von einem Chat-Assistenten das Hallo-Programm aus Beispiel 1.1 schreiben. Bitte ihn dann, es so zu erweitern, dass es dich zusätzlich nach deinem Alter fragt und ausrechnet, wie alt du in zehn Jahren bist. Erkläre danach jede neue Zeile.
4. Frage die KI absichtlich nach etwas, das es nicht gibt (zum Beispiel „Zeig mir die Python-Funktion `datei.zauberhaft_lesen()`“). Wie reagiert sie?

Lösungshinweis

Zu 4: Gute Assistenten sagen, dass die Funktion nicht existiert, und bieten eine Alternative an. Manchmal erfinden sie jedoch eine Erklärung. Genau deshalb testest du Code immer aus. Wenn du im Terminal `python3 -c "help(open)"` aufrufst, siehst du die echte Dokumentation.

Zusammenfassung

Zusammenfassung

- Vibe Coding bedeutet, Software durch Beschreibung in natürlicher Sprache mit einer KI zu bauen und iterativ zu verbessern.

- Der Begriff wurde 2025 durch Andrej Karpathy populär; das Spektrum reicht von „Code nicht lesen“ bis zu sorgfältig geprüfter KI-Unterstützung.
- Deine Rolle: Auftraggeber, Tester und Lerner – nicht Tipper.
- KI-Modelle berechnen wahrscheinliche Fortsetzungen: Sie klingen sicher, können aber halluzinieren und kennen neuere Entwicklungen eventuell nicht.
- Es gibt drei Arten von Assistenten: Chat, Editor-Integration und Terminal-Agenten.
- Je höher der mögliche Schaden, desto weniger darfst du ungeprüft vertrauen.

Kapitel 2: Setup der Umgebung unter Linux

Lernziele

- Du richtest einen Arbeitsordner, Python und Terminal-Werkzeuge ein.
- Du installierst einen Editor (VS Code oder Neovim) und kennst die Alternativen.
- Du wählst einen KI-Assistenten und weißt, wie ein API-Key funktioniert.
- Du legst dein erstes Git-Repository an und machst deinen ersten Commit.

2.1 Die Grundausstattung

Eine gute Vibe-Coding-Umgebung besteht aus fünf Bausteinen:

1. **Terminal** – hast du schon.
2. **Programmiersprache** – wir nutzen Python (unter Linux meist vorinstalliert) und später etwas HTML/JavaScript.
3. **Editor** – zum Lesen und Anpassen von Code.
4. **KI-Assistent** – dein Co-Autor.
5. **Git** – dein Sicherheitsnetz.

Systemcheck

Prüfe zuerst, was bereits vorhanden ist:

Beispiel 2.1: Vorhandene Werkzeuge prüfen

```
$ python3 --version
Python 3.12.3
$ git --version
git version 2.43.0
$ which curl jq tree
/usr/bin/curl
/usr/bin/jq
```

Wenn ein Programm fehlt, fehlt auch seine Ausgabe – wie bei `tree` im Beispiel. Das ist normal; wir installieren es gleich.

Pakete installieren

Debian, Ubuntu und Derivate:

Beispiel 2.2: Grundpakete unter Debian/Ubuntu

```
$ sudo apt update
$ sudo apt install -y git python3 python3-venv python3-pip curl jq tree \
    ripgrep fd-find tmux shellcheck
```

Fedora:

Beispiel 2.3: Grundpakete unter Fedora

```
$ sudo dnf install -y git python3 python3-pip curl jq tree ripgrep fd-find \
    tmux ShellCheck
```

Arch Linux:

Beispiel 2.4: Grundpakete unter Arch

```
$ sudo pacman -S --needed git python python-pip curl jq tree ripgrep fd tmux shellcheck
```

Wofür sind die Werkzeuge gut?

Werkzeug	Zweck
`git`	Versionskontrolle, Sicherheitsnetz
`python3`, `python3-venv`	Sprache und isolierte Umgebungen
`jq`	JSON lesen und filtern
`tree`	Ordnerstruktur anzeigen – ideal, um sie der KI zu zeigen
`ripgrep` (`rg`)	sehr schnelle Textsuche, Nachfolger von `grep -r`
`fd`	schnelle Dateisuche, Nachfolger von `find`
`tmux`	mehrere Terminals in einem Fenster
`shellcheck`	prüft Bash-Skripte auf typische Fehler

> **TIPP – Profi-Trick**
>
> Unter Debian/Ubuntu heißt das Programm `fd` oft `fdfind` (das Paket heißt `fd-find`, das Binary aber anders). Prüfe vor dem Verknüpfen, ob es wirklich da ist und ob der Zielordner existiert und im `PATH` liegt (Beispiel 2.2b). Fehlt `fdfind`, ist die Distribution anders aufgebaut (Fedora und Arch liefern das Programm direkt als `fd`); dann ist kein Symlink nötig.

Beispiel 2.2b: fd/fdfind robust verknüpfen

```
$ command -v fdfind && echo "fdfind gefunden" || echo "fdfind NICHT gefunden"
$ mkdir -p ~/.local/bin
$ echo "$PATH" | tr ':' '\n' | grep -q "$HOME/.local/bin" \
    && echo "im PATH" || echo "NICHT im PATH – siehe Beispiel 2.18"
$ ln -s "$(command -v fdfind)" ~/.local/bin/fd
```

2.2 Der Arbeitsordner

Ordnung im Dateisystem spart später Nerven. Wir legen einen festen Platz für alle Projekte an:

Beispiel 2.5: Arbeitsordner anlegen

```
$ mkdir -p ~/vibe/{projekte,experimente,notizen}
$ cd ~/vibe
$ tree -L 2
.
├── experimente
├── notizen
└── projekte
```

- `projekte/` – Dinge, die du behalten und mit Git verwalten willst.
- `experimente/` – Wegwerf-Versuche.

- `notizen/` – dein Lerntagebuch (siehe Kapitel 3).

2.3 Python und virtuelle Umgebungen

Python-Programme brauchen oft zusätzliche Bibliotheken. Damit sich Projekte nicht gegenseitig stören, bekommt jedes Projekt seine eigene **virtuelle Umgebung** (englisch: virtual environment, kurz *venv*): einen Ordner mit einer privaten Python-Installation.

Beispiel 2.6: Virtuelle Umgebung anlegen und aktivieren

```
$ cd ~/vibe/experimente
$ mkdir test && cd test
$ python3 -m venv .venv
$ source .venv/bin/activate
(.venv) $ which python
/home/DEINNAME/vibe/experimente/test/.venv/bin/python
(.venv) $ pip install requests
(.venv) $ deactivate
$
```

Nach `source .venv/bin/activate` erscheint `(.venv)` vor der Eingabeaufforderung. Alles, was du jetzt mit `pip install` installierst, landet nur in diesem Ordner. Mit `deactivate` verlässt du die Umgebung wieder.

> **WARNUNG – Sicherheitsrisiko**
>
> Installiere Python-Pakete unter Linux nicht mit `sudo pip install` ins System. Das kann Systemwerkzeuge deiner Distribution beschädigen. Viele Distributionen blockieren das inzwischen mit der Meldung „externally-managed-environment“. Die richtige Antwort darauf ist immer: **eine venv verwenden**.

Die Liste der installierten Pakete speicherst du so, damit andere (und du selbst später) sie neu installieren können:

Beispiel 2.7: Abhängigkeiten festhalten und wiederherstellen

```
(.venv) $ pip freeze > requirements.txt
(.venv) $ cat requirements.txt
certifi==...
requests==...
$ pip install -r requirements.txt     # später, in einer neuen venv
```

> **Gut zu wissen**
>
> `pip freeze` schreibt eine **Momentaufnahme** deiner aktuellen venv auf: alle Pakete, die gerade installiert sind, einschließlich der, die du nie direkt angefordert hast, sondern die als Abhängigkeit eines anderen Pakets mitkamen. Welche Zeile du selbst mit `pip install` angefordert hast, steht darin nicht mehr. Für den Einstieg reicht das völlig; wenn ein Projekt wächst, lohnt sich die Trennung zwischen einer kurzen, von dir gepflegten Liste deiner **direkten** Abhängigkeiten (häufig `requirements.in` genannt) und der daraus erzeugten, vollständigen `requirements.txt`. Notiere dir außerdem Python-Version und Betriebssystem im Projekt (z. B. in der README): Eine `requirements.txt` installiert sich nicht überall identisch, wenn sich diese unterscheiden.

2.4 Der Editor

Ein Editor ist dein Fenster in den Code. Du musst ihn nicht perfekt beherrschen, aber du solltest Code lesen, kleine Änderungen vornehmen und Dateien schnell wechseln können.

Option A: Visual Studio Code (empfohlen für Einsteiger)

VS Code ist grafisch, hat eine integrierte Terminal-Leiste und Erweiterungen für alle gängigen KI-Assistenten. Installation je nach Distribution:

Beispiel 2.8: VS Code installieren (Varianten)

```
# Variante 1: .deb-Paket von der Projektseite (Debian/Ubuntu)
$ sudo apt install ./code_*.deb

# Variante 2: Flatpak (distributionsunabhängig)
$ flatpak install flathub com.visualstudio.code

# Variante 3: Paketmanager (Arch, Community-Paket)
$ sudo pacman -S code
```

> **Gut zu wissen**
>
> VS Code von Microsoft enthält Telemetrie und Microsoft-Dienste. Wer das nicht möchte, findet in **VSCodium** einen Build ohne diese Bestandteile; manche Erweiterungen sind dort allerdings nicht oder nur eingeschränkt verfügbar. Für den Einstieg ist beides in Ordnung.

Starte VS Code aus dem Projektordner, damit es den Ordner als Arbeitsbereich öffnet:

Beispiel 2.9: Ordner in VS Code öffnen

```
$ cd ~/vibe/projekte
$ code .
```

Nützliche Tastenkürzel zum Anfang:

Kürzel	Wirkung
`Strg+P`	Datei per Namen öffnen
`Strg+Umschalt+P`	Befehlspalette (alles ist hier auffindbar)
`Strg+ö`	Integriertes Terminal ein/aus
`Strg+Z`	Rückgängig
`Strg+F` / `Strg+Umschalt+F`	Suchen in Datei / im ganzen Projekt

Option B: Neovim

Neovim läuft im Terminal und ist extrem schnell, hat aber eine steile Lernkurve. Wenn du dich fürs Terminal begeisterst, lohnt es sich – für den Anfang aber nicht nötig.

Beispiel 2.10: Neovim installieren und das Tutorial starten

```
$ sudo apt install neovim        # Fedora: sudo dnf install neovim
$ nvim --version | head -1
$ nvim +Tutor                    # interaktives Tutorial, ca. 30 Minuten
```

Die fünf wichtigsten Dinge in Neovim: Mit `i` wechselst du in den **Einfügemodus** (Text tippen), mit `Esc` zurück in den **Normalmodus**. `:w` speichert, `:q` beendet, `:wq` macht beides, `:q!` beendet ohne

Speichern.

> **TIPP – Profi-Trick**
>
> Wer nur kurz eine Konfigurationsdatei ändern will, kommt mit `nano` aus: `nano datei.txt`, speichern mit `Strg+O`, beenden mit `Strg+X`. Das ist völlig legitim.

2.5 KI-Assistenten einrichten

Du hast drei Wege, die auch kombinierbar sind. Beginne mit Weg 1, bis du dich sicher fühlst.

Weg 1: Chat im Browser

Kein Setup nötig: Konto anlegen, Chat öffnen, Code hin- und herkopieren. Vorteil: sehr sicher, weil die KI keinen Zugriff auf dein System hat. Nachteil: viel Kopieren, und die KI sieht nur, was du ihr zeigst.

Weg 2: Erweiterung im Editor

Im VS-Code-Erweiterungsbereich (`Strg+Umschalt+X`) findest du Assistenten wie GitHub Copilot, Continue und weitere. Sie zeigen Vorschläge direkt im Code und können mit der KI über geöffnete Dateien chatten. Anmeldung und Abrechnung laufen je nach Anbieter über ein Konto oder einen eigenen API-Key.

Weg 3: Terminal-Agent

Ein Terminal-Agent läuft in deinem Projektordner, liest Dateien und schlägt Änderungen vor. Beispiele sind Claude Code, Aider, Codex CLI und Gemini CLI. Die genauen Installationsschritte ändern sich, daher gilt: **folge der offiziellen Anleitung**. Bei Claude Code findest du sie in der Dokumentation unter docs.claude.com (Bereich Claude Code); verteilt wird das Programm unter anderem als npm-Paket `@anthropic-ai/claude-code`.

Der typische Ablauf bei jedem Terminal-Agenten ist gleich:

Beispiel 2.11: Typischer Ablauf mit einem Terminal-Agenten

```
$ cd ~/vibe/projekte/mein-projekt     # IMMER erst in den richtigen Ordner wechseln
$ git status                         # sauberer Stand? Sonst erst committen
$ <agent-name>                       # Agent starten (Name je nach Werkzeug)
> Lies bitte die README und erkläre mir, was dieses Projekt tut.
```

> **WARNUNG – Sicherheitsrisiko**
>
> Starte einen Terminal-Agenten **nie** in deinem Home-Verzeichnis (`~`) oder im Wurzelverzeichnis (`/`). Er hat dann Zugriff auf alle deine Dateien, auch auf `~/.ssh` und Passwortdateien. Wechsle immer zuerst in den Projektordner. Erlaube Befehle einzeln und lies sie, bevor du „Ja“ sagst.

Was kostet das?

Die Preismodelle ändern sich häufig: Manche Dienste bieten kostenlose Kontingente, andere Abos mit Nutzungslimits, wieder andere rechnen pro verarbeitetem Text (Token) ab. Prüfe die aktuellen Preise beim Anbieter und setze nach Möglichkeit ein **Ausgabenlimit** im Konto.

2.6 API-Keys – was sie sind und wie du sie ablegst

Ein **API-Key** ist ein langer Zugangsschlüssel (eine Zeichenkette), mit dem ein Programm bei einem Dienst „angemeldet“ ist. Wer den Key hat, kann den Dienst **auf deine Rechnung** nutzen. Behandle ihn wie ein Passwort.

Die sauberste Lösung für den Anfang: Der Key liegt in einer Datei, die nur du lesen darfst, und wird als **Umgebungsvariable** bereitgestellt.

Beispiel 2.12: API-Key sicher ablegen

```
$ mkdir -p ~/.config/vibe
$ nano ~/.config/vibe/keys.env
# Inhalt der Datei (eine Zeile pro Key):
# export ANTHROPIC_API_KEY="hier-dein-key"
$ chmod 600 ~/.config/vibe/keys.env
$ ls -l ~/.config/vibe/keys.env
-rw------- 1 eugen eugen 52 Okt 23 10:12 /home/eugen/.config/vibe/keys.env
```

`chmod 600` bedeutet: Lesen und Schreiben nur für dich, niemand sonst. Bei Bedarf lädst du die Datei in die aktuelle Terminal-Sitzung:

Beispiel 2.13: Key in die Sitzung laden und prüfen

```
$ source ~/.config/vibe/keys.env
$ [ -n "$ANTHROPIC_API_KEY" ] && echo "Key gesetzt" || echo "Key fehlt"
Key gesetzt
```

Beachte: Wir geben den Key nie mit `echo $KEY` aus, damit er nicht auf dem Bildschirm oder im Terminalverlauf landet.

> **WARNUNG – Sicherheitsrisiko**
>
> **Niemals** einen API-Key direkt in eine Programmdatei schreiben, in einen Chat einfügen oder in ein Git-Repository committen. Bots durchsuchen öffentliche Repositories ständig nach Keys; ein veröffentlichter Key ist oft binnen Minuten missbraucht. Mehr dazu in Kapitel 6.

2.7 Git – dein Sicherheitsnetz

Git speichert Schnappschüsse (englisch: *commits*) deines Projekts. Jeder Schnappschuss hat eine Nachricht, und du kannst jederzeit zu einem früheren Stand zurückkehren. Für Vibe Coding ist das Gold wert: Wenn die KI etwas kaputt macht, gehst du einfach zurück.

Einmalige Einrichtung

Beispiel 2.14: Git konfigurieren

```
$ git config --global user.name "Dein Name"
$ git config --global user.email "du@example.org"
$ git config --global init.defaultBranch main
$ git config --global --list
```

Das erste Repository

Beispiel 2.15: Erstes Repository mit erstem Commit

```
$ mkdir -p ~/vibe/projekte/git-test && cd ~/vibe/projekte/git-test
$ git init
$ echo "# Mein erstes Projekt" > README.md
$ git status
$ git add README.md
$ git commit -m "Erster Commit: README angelegt"
$ git log --oneline
a1b2c3d Erster Commit: README angelegt
```

Der Kreislauf besteht aus drei Schritten:

1. Du änderst Dateien (**Arbeitsverzeichnis**).
2. Mit `git add` markierst du, welche Änderungen in den nächsten Schnappschuss sollen (**Staging-Bereich**).
3. Mit `git commit` speicherst du den Schnappschuss (**Repository**).

Die zehn Befehle, die du wirklich brauchst

Befehl	Wirkung
`git status`	Was hat sich geändert?
`git diff`	Zeige die Änderungen im Detail
`git add -A`	Alle Änderungen für den Commit vormerken
`git commit -m "Text"`	Schnappschuss speichern
`git log --oneline`	Verlauf kompakt anzeigen
`git restore datei`	Datei auf den letzten Commit zurücksetzen
`git switch -c name`	Neuen Zweig (Branch) anlegen und wechseln
`git switch main`	Zurück zum Hauptzweig
`git stash`	Änderungen kurz beiseitelegen
`git revert <id>`	Einen Commit durch einen Gegen-Commit rückgängig machen

Die .gitignore-Datei

Manche Dateien gehören nicht ins Repository: Keys, virtuelle Umgebungen, temporäre Dateien. Du schließt sie in einer Datei namens `.gitignore` aus:

Beispiel 2.16: Eine sinnvolle .gitignore für Python-Projekte

```
$ cat > .gitignore << 'EOF'
.venv/
__pycache__/
*.pyc
.env
*.env
*.log
EOF
$ git add .gitignore && git commit -m "gitignore hinzugefügt"
```

> **TIPP – Profi-Trick**

Die Vibe-Coding-Regel Nr. 1: Committe **vor jeder größeren KI-Änderung**. Dann kostet jeder Fehlschlag nur einen Befehl: `git restore .` verwirft alle Änderungen seit dem letzten Commit. Wer beliebig experimentieren will, nutzt einen eigenen Branch: `git switch -c experiment`.

WARNUNG – Sicherheitsrisiko

`git restore .` und `git reset --hard` löschen nicht committete Änderungen **endgültig**. Prüfe vorher mit `git status` und `git diff`, was du verlierst.

2.8 Projektvorlage zum Wiederverwenden

Zum Abschluss ein kleines Skript, das neue Projekte mit der richtigen Grundstruktur anlegt. Du kannst es selbst abtippen oder dir von der KI erklären lassen.

Beispiel 2.17: Skript "neues-projekt" zum Anlegen neuer Projekte

```
#!/usr/bin/env bash
# Legt ein neues Projekt mit Git, venv und .gitignore an.
set -euo pipefail

if [ $# -ne 1 ]; then
    echo "Benutzung: $0 PROJEKTNAME" >&2
    exit 1
fi
# Nur einfache Namen erlauben: Buchstaben, Ziffern, _ und -, nicht leer.
# Das verhindert, dass z. B. "../anderer-ordner" das vorgesehene
# Projekteverzeichnis verlässt (Pfad-Traversal, siehe Kapitel 6.4).
if ! [[ "$1" =~ ^[A-Za-z0-9][A-Za-z0-9_-]*$ ]]; then
    echo "Fehler: '$1' ist kein gültiger Projektname (nur Buchstaben, Ziffern, _ und -)." >&2
    exit 1
fi
ziel="$HOME/vibe/projekte/$1"
if [ -e "$ziel" ]; then
    echo "Fehler: $ziel existiert bereits." >&2
    exit 1
fi

mkdir -p "$ziel"
cd "$ziel"
git init -q
python3 -m venv .venv
printf '.venv/\n__pycache__/\n*.pyc\n.env\n*.env\n*.log\n' > .gitignore
printf '# %s\n\nKurzbeschreibung folgt.\n' "$1" > README.md
git add -A && git commit -q -m "Projektgerüst angelegt"
echo "Projekt angelegt: $ziel"
```

Speichere es als `~/.local/bin/neues-projekt`, mache es ausführbar und teste es:

Beispiel 2.18: Skript installieren und ausprobieren

```
$ mkdir -p ~/.local/bin
$ nano ~/.local/bin/neues-projekt          # Inhalt aus Beispiel 2.17 einfügen
$ chmod +x ~/.local/bin/neues-projekt
$ neues-projekt demo
Projekt angelegt: /home/eugen/vibe/projekte/demo
$ neues-projekt ../anderer-ordner
Fehler: '../anderer-ordner' ist kein gültiger Projektname (nur Buchstaben, Ziffern, _ und -).
```

Erklärung der wichtigsten Zeilen: `set -euo pipefail` lässt das Skript bei Fehlern abbrechen, statt weiterzumachen. `$#` ist die Anzahl der übergebenen Argumente, `$1` das erste. Die Zeile mit `=~` prüft den

Namen gegen ein **reguläres Muster** (nur Buchstaben, Ziffern, _ und `-`, beginnend mit einem alphanumerischen Zeichen) und weist alles andere zurück – auch Namen mit `/` oder `..`, die sonst aus dem vorgesehenen Projektordner herausführen könnten. `[ -e "$ziel" ]` prüft, ob etwas unter diesem Pfad existiert. Die Anführungszeichen um Variablen schützen vor Problemen bei Leerzeichen im Namen.

TIPP – Profi-Trick

Diese Art Prüfung – **erst die Eingabe gegen ein enges, erlaubtes Muster validieren, dann erst damit arbeiten** – ist ein allgemeines Prinzip, das weit über dieses Skript hinaus gilt: Lieber genau festlegen, was erlaubt ist (Allowlist), als zu versuchen, einzelne gefährliche Zeichen zu verbieten (Denylist).

Übungen

Übung

1. Lege mit Beispiel 2.5 deinen Arbeitsordner an und prüfe das Ergebnis mit `tree`.
2. Erstelle eine venv in `~/vibe/experimente/test`, installiere `requests`, erzeuge `requirements.txt` und lösche danach die venv. Stelle sie aus `requirements.txt` wieder her.
3. Initialisiere ein Git-Repository, lege drei Commits mit sinnvollen Nachrichten an und sieh dir den Verlauf mit `git log --oneline` an.
4. Ändere eine Datei, schaue die Änderung mit `git diff` an und verwirf sie mit `git restore datei`. **Wichtig:** `git restore` macht nur Änderungen rückgängig, die noch nicht committet sind – teste das deshalb nur in einem frischen, extra dafür angelegten Repository, nie an Dateien, an denen du gerade uncommittete Arbeit hast, die dir wichtig ist.
5. Lege (ohne echten Key) die Datei `keys.env` mit Rechten `600` an und prüfe mit `ls -l`.
6. Teste `neues-projekt` aus Beispiel 2.17/2.18 einmal mit einem gültigen Namen und einmal mit `../test`. Prüfe, dass der zweite Aufruf mit einer Fehlermeldung abbricht und **kein** Ordner außerhalb von `~/vibe/projekte/` entsteht.

Lösungshinweis

Zu 2: Nach dem Löschen von `.venv` legst du mit `python3 -m venv .venv` eine neue an, aktivierst sie und führst `pip install -r requirements.txt` aus. Zu 5: Die Ausgabe von `ls -l` muss mit `-rw-------` beginnen. Zu 6: Der zweite Aufruf meldet „kein gültiger Projektname“ und bricht ab, bevor `mkdir` überhaupt erreicht wird.

Zusammenfassung

Zusammenfassung

- Deine Umgebung besteht aus Terminal, Python, Editor, KI-Assistent und Git.
- Jedes Python-Projekt bekommt eine eigene virtuelle Umgebung (`python3 -m venv .venv`) – niemals `sudo pip install`.
- VS Code ist der bequemste Einstieg; Neovim und nano sind legitime Alternativen.
- Beginne mit einem Chat-Assistenten; wechsle zu Editor-Integration oder Terminal-Agenten, wenn du Routine hast.

- API-Keys liegen in einer Datei mit Rechten `600`, nie im Code und nie im Repository.
- Git ist dein Sicherheitsnetz: committen vor jeder größeren KI-Änderung.

Kapitel 3: Der Vibe-Coding-Workflow

Lernziele

- Du kennst den Kreislauf aus Beschreiben, Erzeugen, Testen, Korrigieren und Sichern.
- Du zerlegst eine Idee in kleine, testbare Schritte.
- Du setzt die KI zum Debuggen ein und gibst ihr den richtigen Kontext.
- Du führst ein Lerntagebuch und eine Projekt-Notizdatei für die KI.

3.1 Der Grundkreislauf

Erfolgreiches Vibe Coding folgt immer demselben Rhythmus. Wer ihn verinnerlicht, vermeidet 90 Prozent aller Frustmomente:

1. **Ziel klären:** Was soll am Ende passieren? (Ein Satz.)
2. **Kleinen Schritt beschreiben:** Nicht „baue mir eine App“, sondern „lies eine Datei und zähle die Zeilen“.
3. **Code erzeugen lassen.**
4. **Ausführen und testen:** Tut es, was es soll?
5. **Bei Erfolg: sichern** (`git commit`). Bei Fehler: Meldung samt Kontext zurückgeben.
6. **Nächster kleiner Schritt.**

Der wichtigste Satz dieses Kapitels: **Je kleiner der Schritt, desto besser das Ergebnis.** Große Aufträge produzieren große, schwer prüfbare Codemengen, in denen sich Fehler verstecken. Kleine Aufträge liefern kleine Änderungen, die du verstehen und testen kannst.

Die Faustregel „Ein Schritt = ein Test“

Nach jedem Schritt solltest du mit einem einzigen Befehl prüfen können, ob er funktioniert hat. Wenn dir kein Test einfällt, ist der Schritt zu groß oder zu unklar.

Zu groß	Besser
„Baue eine Aufgabenverwaltung mit Login und Datenbank.“	„Schreibe ein Programm, das eine Aufgabe in eine Textdatei schreibt.“
„Mach das Skript schneller.“	„Miss, welcher Teil am längsten dauert, und zeige mir die drei langsamsten Stellen.“
„Repariere alle Fehler.“	„Hier ist die Fehlermeldung. Erkläre die Ursache, bevor du etwas änderst.“

3.2 Vor dem ersten Prompt: Plan und Projektnotiz

Bevor du Code erzeugen lässt, nimm dir fünf Minuten für einen Plan. Du kannst dir dabei von der KI helfen lassen, aber **entscheide selbst** über den Umfang.

Beispiel 3.1: Planungs-Prompt – erst planen, dann bauen

```
Ich möchte ein Kommandozeilen-Tool für Linux bauen, das doppelte Dateien in einem
Ordner findet. Ich bin Anfänger. Bitte schreibe noch KEINEN Code.
Schlage mir stattdessen vor:
1. eine minimale Version (MVP), die in einer Stunde machbar ist,
2. eine Liste von 5 kleinen Schritten, jeweils mit einem Testbefehl,
3. was ich bewusst weglasse.
```

Die Antwort sicherst du in einer Projektdatei. Diese Datei dient dir **und** der KI als Gedächtnis, denn Assistenten vergessen zwischen Gesprächen alles (und bei langen Gesprächen auch Details vom Anfang).

Beispiel 3.2: Projektnotiz PROJEKT.md als Gedächtnis

```
# Projekt: dupfinder

## Ziel
Findet doppelte Dateien in einem Ordner (gleicher Inhalt, egal welcher Name).

## Rahmen
- Sprache: Python 3, nur Standardbibliothek
- Läuft auf Linux im Terminal: `python3 dupfinder.py ORDNER`
- Löscht NIE automatisch etwas

## Stand
- [x] Schritt 1: Dateien im Ordner auflisten
- [ ] Schritt 2: Hash pro Datei berechnen
- [ ] Schritt 3: Gleiche Hashes gruppieren
- [ ] Schritt 4: Ausgabe formatieren
## Entscheidungen
- Hash-Verfahren: SHA-256 (sicher genug, in der Standardbibliothek)
```

Diese Datei legst du ins Projektverzeichnis und committest sie mit. Viele Terminal-Agenten lesen sogar automatisch eine Projektdatei mit Regeln (zum Beispiel `CLAUDE.md` bei Claude Code oder `AGENTS.md` bei anderen Werkzeugen). Auch bei einem Chat-Assistenten kannst du sie einfach am Gesprächsbeginn einfügen.

> **TIPP – Profi-Trick**
>
> Beginne jedes neue Gespräch mit demselben Block: Projektziel, Sprache, Rahmenbedingungen („nur Standardbibliothek“, „keine neuen Abhängigkeiten ohne Rückfrage“), aktueller Stand. Das spart dir endlose Wiederholungen und verhindert, dass die KI plötzlich ein riesiges Framework einbaut.

3.3 Code erzeugen, lesen, ausführen

Schritt 1 am Beispiel dupfinder

Prompt:

Beispiel 3.3: Prompt für einen kleinen, klaren Schritt

```
Schreibe dupfinder.py. Das Programm bekommt einen Ordnerpfad als Argument und
gibt alle Dateien darin (rekursiv) mit Pfad und Größe in Bytes aus.
Nur Standardbibliothek. Wenn der Ordner nicht existiert, eine klare
Fehlermeldung ausgeben und mit Exit-Code 1 beenden.
```

Ergebnis (typisch):

Beispiel 3.4: dupfinder.py, Schritt 1 – Dateien auflisten

```
import sys
from pathlib import Path


def alle_dateien(ordner: Path):
    """Liefert alle regulären Dateien unterhalb von ordner."""
    for pfad in ordner.rglob("*"):
        if pfad.is_file():
            yield pfad


def main():
    if len(sys.argv) != 2:
        print("Benutzung: python3 dupfinder.py ORDNER", file=sys.stderr)
        sys.exit(2)
    ordner = Path(sys.argv[1])
    if not ordner.is_dir():
        print(f"Fehler: '{ordner}' ist kein Ordner.", file=sys.stderr)
        sys.exit(1)
    for datei in alle_dateien(ordner):
        print(f"{datei.stat().st_size:>12}  {datei}")


if __name__ == "__main__":
    main()
```

So liest du Code, ohne Programmierer zu sein

Du musst keine Zeile auswendig kennen. Lies von **oben nach unten** und achte auf diese Bausteine:

- `import` holt fertige Werkzeugkästen (hier `sys` für Argumente, `Path` für Dateipfade).
- `def name(...)` definiert eine **Funktion**, einen benannten Baustein. `main()` ist hier der Startpunkt.
- `if ...:` sind Bedingungen („wenn dies, dann das“). Der eingerückte Block darunter gehört dazu.
- `for ... in ...:` ist eine Schleife: etwas für jedes Element wiederholen.
- `print(...)` gibt Text aus; `file=sys.stderr` schreibt in den Fehlerkanal.
- `sys.exit(1)` beendet das Programm mit Exit-Code 1 (Fehler). Dieselbe Konvention kennst du von der Shell: 0 bedeutet Erfolg.

Wichtig bei Python: **Einrückung ist Syntax**. Vier Leerzeichen zeigen, was zu welchem Block gehört.

> **Gut zu wissen**
>
> Diese erste Fassung behandelt Symlinks (Verknüpfungen auf andere Dateien oder Ordner) noch nicht gesondert – sie folgt `rglob()` überallhin, auch in verlinkte Unterordner. Das reicht für den Einstieg. Die vollständige Fassung in Kapitel 5 schließt das bewusst aus; dort erfährst du auch, warum eine naheliegende Lösung (nur `is_symlink()` auf die gefundene Datei prüfen) dafür allein nicht reicht.

Ausführen und beobachten

Beispiel 3.5: Schritt 1 testen

```
$ cd ~/vibe/projekte/dupfinder
$ python3 dupfinder.py ~/vibe/notizen
         142  /home/eugen/vibe/notizen/tagebuch.md
$ python3 dupfinder.py /gibt/es/nicht
Fehler: '/gibt/es/nicht' ist kein Ordner.
$ echo $?
1
```

Du hast gerade drei Dinge getestet: den Normalfall, den Fehlerfall und den Exit-Code. Das ist **mehr Qualitätssicherung, als viele Einsteiger je betreiben**.

Danach folgt der Commit:

Beispiel 3.6: Funktionierenden Stand sichern

```
$ git add -A
$ git commit -m "dupfinder: Dateien rekursiv auflisten"
```

3.4 Iterieren: Änderungswünsche formulieren

Nach dem ersten Erfolg wird der Code Schritt für Schritt erweitert. Immer mit dem aktuellen Code im Gespräch und einer klaren Eingrenzung:

Beispiel 3.7: Erweiterung mit klarer Eingrenzung

```
Hier ist mein aktuelles dupfinder.py (siehe unten). Erweitere es um Schritt 2:
Berechne für jede Datei den SHA-256-Hash (in Blöcken von 1 MiB, damit große
Dateien nicht komplett in den Speicher geladen werden) und gib statt der
Größe den Hash aus. Ändere sonst nichts am Programm. Zeige mir nur die
geänderten Funktionen, nicht die ganze Datei.

[Code einfügen]
```

Die Formulierungen „Ändere sonst nichts“ und „Zeige nur die geänderten Funktionen“ sind kleine, aber mächtige Hebel: Sie verhindern, dass die KI unaufgefordert umbaut.

Änderungen prüfen mit git diff

Wenn du die Änderung übernommen hast, schau dir an, **was sich wirklich geändert hat**:

Beispiel 3.8: Änderungen der KI mit git diff kontrollieren

```
$ git diff
@@ -5,6 +5,13 @@ from pathlib import Path
+import hashlib
+
+def sha256_von(datei: Path) -> str:
+    h = hashlib.sha256()
+    with datei.open("rb") as f:
+        for block in iter(lambda: f.read(1024 * 1024), b""):
+            h.update(block)
+    return h.hexdigest()
```

Zeilen mit `+` sind neu, Zeilen mit `-` wurden entfernt. Selbst als Anfänger erkennst du so schnell, ob die KI nur das Gewünschte getan hat oder auch Unerwartetes: gelöschte Funktionen, geänderte Dateinamen, neue Importe.

> **TIPP – Profi-Trick**
>
> Ein Blick auf `git diff --stat` zeigt dir in einer Zeile pro Datei, wie viel sich geändert hat. Wenn du nur „eine Funktion ergänzt“ hast erwarten lassen und 400 Zeilen geändert wurden, stimmt etwas nicht.

3.5 Debugging mit KI

Debugging heißt, Fehler zu finden und zu beheben. Es ist die Disziplin, in der KI-Assistenten besonders glänzen – wenn du ihnen die richtigen Informationen gibst.

Die vier Bestandteile einer guten Fehlermeldung an die KI

1. **Was wolltest du tun?** (Ziel)
2. **Was hast du getan?** (exakter Befehl)
3. **Was ist passiert?** (komplette Fehlermeldung, nicht nur die letzte Zeile)
4. **Welche Umgebung?** (Distribution, Python-Version, relevante Dateien)

Beispiel 3.9: Vorlage für eine Fehleranfrage

```
Ziel: dupfinder soll alle Dateien in ~/Bilder hashen.
Befehl: python3 dupfinder.py ~/Bilder
Fehler (komplett):
Traceback (most recent call last):
  File "dupfinder.py", line 24, in <module>
    main()
  File "dupfinder.py", line 20, in main
    h = sha256_von(datei)
PermissionError: [Errno 13] Permission denied: '/home/eugen/Bilder/geheim.jpg'
Umgebung: Debian 12, Python 3.11.2.
Erkläre mir zuerst die Ursache in zwei Sätzen. Schlage dann eine minimale
Korrektur vor, die das Programm bei nicht lesbaren Dateien nicht abstürzen lässt.
```

Du kannst die Fehlerausgabe direkt aus dem Terminal kopieren. Wenn du sie in eine Datei schreiben willst, hilft die Umleitung beider Kanäle:

Beispiel 3.10: Fehlerausgabe in eine Datei umleiten

```
$ python3 dupfinder.py ~/Bilder > ausgabe.txt 2> fehler.txt
$ python3 dupfinder.py ~/Bilder 2>&1 | tee lauf.log
```

Mit `2>` leitest du den Fehlerkanal um, mit `2>&1` führst du ihn mit der normalen Ausgabe zusammen. `tee` zeigt die Ausgabe am Bildschirm und schreibt sie zugleich in die Datei.

Die Debug-Schleife

1. Fehlermeldung samt Kontext an die KI geben.
2. **Ursache erklären lassen**, bevor ein Fix kommt. Das verhindert blinde Pflaster.
3. Fix einspielen, erneut testen.
4. Tritt derselbe Fehler nach zwei Versuchen weiter auf: **stopp**. Wechsle die Strategie (siehe Abschnitt 3.7).

Eigene Diagnose-Ausgaben

Manchmal weißt du nicht, was im Programm vorgeht. Bitte die KI, **Diagnose-Ausgaben** einzubauen (`print` oder das Modul `logging`) und starte erneut. So siehst du, welche Werte wirklich vorliegen:

Beispiel 3.11: Diagnose-Ausgaben mit logging

```
import logging

logging.basicConfig(level=logging.DEBUG, format="%(levelname)s: %(message)s")

def verarbeite(datei):
    logging.debug("Verarbeite %s", datei)
    # ...
```

3.6 Kontext-Management

Ein KI-Assistent kann nur so viel „im Kopf“ behalten, wie in sein **Kontextfenster** passt, also die Menge an Text, die er pro Gespräch gleichzeitig berücksichtigt. Das Fenster ist heute oft sehr groß, aber nicht unendlich, und die Qualität sinkt, wenn Gespräche extrem lang und unübersichtlich werden.

Symptome eines überladenen Kontexts

- Die KI „vergisst“ Vorgaben vom Anfang (zum Beispiel „nur Standardbibliothek“).
- Sie wiederholt Fehler, die ihr schon erklärt wurden.
- Sie ändert Dinge, die du nicht angefasst haben wolltest.

Gegenmittel

Maßnahme	Wie
Neues Gespräch beginnen	Bei neuem Thema oder nach mehr als etwa 30 Nachrichten
Zusammenfassung anfordern	„Fasse Projektstand und Entscheidungen in 10 Zeilen zusammen“ – in die neue Sitzung einfügen
Projektnotiz pflegen	PROJEKT.md aktuell halten (Beispiel 3.2)
Nur Relevantes zeigen	Nicht das ganze Projekt, sondern die betroffene Datei und die Fehlermeldung
Struktur zeigen	`git ls-files` oder `tree -L 2` an die KI geben

Beispiel 3.12: Projektstruktur kompakt für die KI ausgeben

```
$ tree -L 2 -I '.venv|__pycache__|.git'
.
├── PROJEKT.md
├── README.md
├── dupfinder.py
└── tests
    └── test_dupfinder.py
```

Dateien gezielt zeigen

Schicke nicht ganze Projekte hin, sondern gezielt Auszüge:

Beispiel 3.13: Gezielte Auszüge statt ganzer Dateien

```
$ sed -n '15,40p' dupfinder.py          # nur Zeilen 15 bis 40
$ rg -n "def " dupfinder.py             # Übersicht aller Funktionen mit Zeilennummer
$ git diff HEAD~1                       # nur die letzte Änderung
```

WARNUNG – Sicherheitsrisiko

Bevor du Dateien in einen Chat kopierst, prüfe, dass **keine Zugangsdaten** darin stehen: API-Keys, Passwörter, Tokens, private Adressen. Suche vorher mit `rg -i "key|token|passwort|secret" datei`.

3.7 Wenn nichts mehr geht: Notausgänge

Irgendwann steckst du fest: Die KI dreht sich im Kreis, jeder Fix erzeugt zwei neue Fehler. Hier sind deine Notausgänge, nach Aufwand sortiert:

1. **Zurück zum letzten guten Stand:** `git restore .` oder `git stash`. Danach den Schritt **kleiner** neu formulieren.
2. **Neues Gespräch:** Nur Projektnotiz, aktuelle Datei und die Fehlermeldung einfügen.
3. **Problem isolieren:** Ein winziges Testprogramm, das nur den Fehler reproduziert (die KI kann es schreiben).
4. **Anderen Assistenten fragen:** Eine zweite Meinung hilft oft.
5. **Verständnisfrage statt Reparaturbitte:** „Erkläre mir, was dieser Programmteil tun soll.“ Oft liegt das Problem in einem Missverständnis über die Aufgabe.
6. **Pause machen.** Ernsthaft.

3.8 Das Lerntagebuch

Eine Datei `~/vibe/notizen/tagebuch.md` kostet zwei Minuten pro Sitzung und macht dich schnell besser:

Beispiel 3.14: Lerntagebuch-Eintrag

```
## 2026-10-23 – dupfinder
- Gebaut: Dateien auflisten, Hash berechnen
- Neu gelernt: `with open(...)` schließt Dateien automatisch
- Fehler: PermissionError -> mit try/except abgefangen
- Frage offen: Warum liest man in Blöcken statt alles auf einmal?
- Nächstes Mal: Gruppieren nach Hash
```

Die offene Frage ist der wertvollste Teil: Stelle sie in der nächsten Sitzung der KI. So wächst dein Verständnis, ohne dass du je ein Lehrbuch durcharbeiten musst.

Übungen

Übung

1. Plane ein eigenes kleines Tool (zum Beispiel „sortiere Downloads nach Dateityp“). Lass die KI nur einen Plan mit fünf Schritten und je einem Testbefehl erstellen – ohne Code.
2. Setze Schritt 1 deines Plans um. Teste Normal- und Fehlerfall und committe.
3. Baue absichtlich einen Fehler ein (zum Beispiel einen falschen Dateipfad) und formuliere die Fehleranfrage nach Beispiel 3.9.
4. Lass die KI eine Änderung machen und kontrolliere sie mit `git diff --stat` und `git diff`. Hat sie mehr geändert als gewünscht?
5. Lege `PROJEKT.md` für dein Tool an und beginne ein neues Gespräch mit dieser Datei.

Lösungshinweis

Zu 3: Eine gute Anfrage nennt Ziel, Befehl, die vollständige Fehlermeldung und die Umgebung und bittet zuerst um eine Erklärung der Ursache. Zu 4: Falls die KI zusätzliche Dateien oder Funktionen verändert hat, ergänze beim nächsten Mal „Ändere sonst nichts“ und prüfe den Diff erneut.

Zusammenfassung

Zusammenfassung

- Der Kreislauf: Ziel klären, kleinen Schritt beschreiben, erzeugen, testen, sichern, wiederholen.
- Ein Schritt = ein Test. Wenn du nicht testen kannst, ist der Schritt zu groß.
- Eine Projektnotiz (`PROJEKT.md`) ist das Gedächtnis von dir und der KI.
- Mit `git diff` prüfst du jede KI-Änderung; committe jeden funktionierenden Stand.
- Gute Fehleranfragen enthalten Ziel, Befehl, vollständige Fehlermeldung und Umgebung; lass zuerst die Ursache erklären.
- Halte den Kontext schlank: neue Gespräche, Zusammenfassungen, gezielte Auszüge.
- Notausgänge: zurücksetzen, neu starten, isolieren, zweite Meinung, Pause.

Kapitel 4: Prompting-Grundlagen für Code

Lernziele

- Du formulierst Anweisungen präzise, vollständig und überprüfbar.
- Du lieferst Kontext, ohne die KI zu überfluten.
- Du speist Fehlermeldungen und Logs richtig ein.
- Du nutzt die KI für Code-Reviews und Verständnisfragen.

4.1 Was einen guten Prompt ausmacht

Ein **Prompt** ist deine Anweisung an die KI. Die Qualität des Codes hängt stark von der Qualität des Prompts ab. Ein guter Code-Prompt beantwortet sechs Fragen:

Frage	Beispiel
Was soll gebaut werden?	„Ein Python-Skript, das Logdateien auswertet“
Wofür / warum?	„Ich will sehen, welche IP-Adressen am häufigsten fehlschlagen“
Womit (Technik)?	„Python 3, nur Standardbibliothek, läuft auf Debian“
Wie soll es sich verhalten?	„Aufruf: `python3 auswerten.py LOGDATEI`; Ausgabe als Tabelle“
Was nicht?	„Keine Dateien verändern, keine Netzwerkzugriffe“
Woran erkenne ich Erfolg?	„Bei dieser Beispieldatei soll Zeile X erscheinen“

Vorher / Nachher

Schwacher Prompt:

Beispiel 4.1: Ein zu vager Prompt

```
Mach mir ein Skript für meine Logs.
```

Starker Prompt:

Beispiel 4.2: Ein präziser Prompt

```
Schreibe ein Python-3-Skript (nur Standardbibliothek) für Linux.
Aufgabe: Es liest die Datei /var/log/auth.log (Pfad per Argument änderbar)
und zählt, wie oft pro IP-Adresse der Text "Failed password" vorkommt.
Ausgabe: die 10 häufigsten IP-Adressen als Tabelle (IP, Anzahl), absteigend.
Randfälle: Datei nicht lesbar -> verständliche Meldung und Exit-Code 1.
Nicht erlaubt: Dateien ändern, Netzwerkzugriffe, externe Pakete.
Erkläre am Ende in drei Sätzen, wie das Skript funktioniert.
```

Der zweite Prompt kostet 30 Sekunden mehr Tipparbeit und spart eine halbe Stunde Hin und Her.

4.2 Bewährte Muster

Muster 1: Rolle und Zielgruppe

„Erkläre mir das, als wäre ich Linux-Anwender ohne Programmierkenntnisse.“ Diese eine Zeile verändert die Erklärungen drastisch: weniger Fachbegriffe, mehr Vergleiche.

Muster 2: Erst verstehen, dann bauen

Beispiel 4.3: Rückfragen erzwingen

```
Bevor du Code schreibst: Stelle mir bis zu fünf Rückfragen, falls meine
Beschreibung etwas offen lässt. Wenn alles klar ist, sage "Alles klar".
```

Dieses Muster fängt Missverständnisse ab, bevor sie in Code stecken.

Muster 3: Beispiele geben

Zeige Eingabe und gewünschte Ausgabe, denn nichts ist eindeutiger:

Beispiel 4.4: Mit Beispiel-Ein- und Ausgabe arbeiten

```
Das Programm soll Dateinamen aufräumen. Beispiele:
  "Urlaub 2024 (final).JPG"  ->  "urlaub_2024_final.jpg"
  "  Mein Foto!!.png"         ->  "mein_foto.png"
Regeln: Kleinbuchstaben, Leerzeichen und Sonderzeichen zu "_", Endung klein.
```

Muster 4: Format vorgeben

„Zeige nur das Diff.“ – „Antworte mit einer Tabelle.“ – „Gib erst den Plan, dann den Code in einem einzigen Block.“

Muster 5: Grenzen setzen

Schreibe Einschränkungen ausdrücklich hin: „Keine neuen Abhängigkeiten.“ – „Ändere keine anderen Funktionen.“ – „Lösche nichts, ohne zu fragen.“

Muster 6: Selbstprüfung verlangen

Beispiel 4.5: Selbstprüfung anfordern

```
Prüfe deinen Code vor der Antwort auf: Randfälle (leere Datei, fehlende Rechte),
Pfade mit Leerzeichen und mögliche Datenverluste. Nenne mir, was du geprüft hast.
```

> **Gut zu wissen**
>
> Selbstprüfung verbessert Ergebnisse oft, ersetzt aber **nie** deinen eigenen Test. Die KI kann behaupten, geprüft zu haben, ohne den Code je auszuführen (sofern sie keine Ausführungsumgebung hat).

4.3 Kontext liefern: Was, wie viel, in welcher Form

Kontext ist das, was die KI über dein Projekt wissen muss. Zu wenig führt zu Raten, zu viel zu Verwirrung. Eine praktische Reihenfolge:

1. **Zielbeschreibung** (zwei Sätze)
2. **Umgebung** (Distribution, Sprache und Version, wichtige Pakete)
3. **Relevanter Code** (die betroffene Funktion oder Datei, nicht das ganze Projekt)

4. **Konkretes Problem oder Wunsch**
5. **Ausgabeformat**

Die Umgebung ermittelst du mit wenigen Befehlen:

Beispiel 4.6: Umgebungsinfos für die KI sammeln

```
$ cat /etc/os-release | head -3
$ python3 --version
$ pip list 2>/dev/null | head -20
$ uname -m            # Prozessor-Architektur, z. B. x86_64 oder aarch64
```

Kopiere Code in **Codeblöcke**, damit Struktur und Einrückung erhalten bleiben. Bei Dateien mit Namen schreibe den Namen davor:

Beispiel 4.7: Mehrere Dateien sauber übergeben

```
Datei: dupfinder.py
(Inhalt)

Datei: tests/test_dupfinder.py
(Inhalt)
```

4.4 Fehlermeldungen richtig einspeisen

Die Fehlermeldung ist der wertvollste Hinweis, den die KI bekommen kann. Gib sie **vollständig und unverändert** weiter.

Eine Fehlermeldung lesen lernen

Python-Fehler (**Tracebacks**) liest du **von unten nach oben**: Die letzte Zeile nennt die Fehlerart und die Ursache, darüber steht der Weg dorthin.

Beispiel 4.8: Traceback richtig lesen

```
Traceback (most recent call last):
  File "auswerten.py", line 31, in <module>      <- 3. wo wurde gestartet
    main()
  File "auswerten.py", line 22, in main          <- 2. aus welcher Funktion
    zahlen = [int(z) for z in zeile.split()]
ValueError: invalid literal for int() with base 10: 'abc'   <- 1. DAS ist das Problem
```

Hier steht: Der Text „abc“ ließ sich nicht in eine ganze Zahl umwandeln. Die Ursache liegt in den **Daten** (eine Zeile enthält Text), nicht im Programm selbst.

Bash-Fehler

Beispiel 4.9: Typische Shell-Fehler und ihre Bedeutung

```
bash: ./skript.sh: Permission denied          -> Datei nicht ausführbar: chmod +x
bash: ./skript.sh: /bin/bash^M: bad interpreter -> Windows-Zeilenenden: dos2unix
command not found: jq                          -> Programm nicht installiert
syntax error near unexpected token `fi'        -> Fehler in if/then/fi-Struktur
```

Logs und lange Ausgaben kürzen

Bei langen Logs sende nicht alles, sondern die Umgebung der ersten Fehlermeldung:

Beispiel 4.10: Relevante Logzeilen herausfiltern

```
$ journalctl -u mein-dienst --since "10 min ago" | head -50
$ grep -n -B3 -A10 "Traceback" lauf.log | head -60
$ tail -n 40 lauf.log
```

`-B3` und `-A10` zeigen drei Zeilen davor und zehn danach. Entferne vor dem Senden persönliche Daten wie Benutzernamen, IP-Adressen oder Tokens.

> **TIPP – Profi-Trick**
>
> Die schnellste Debug-Anfrage der Welt: Fehlermeldung kopieren, einfügen, dazuschreiben „Was bedeutet das, und was ist die wahrscheinlichste Ursache? Bitte erst erklären, noch nichts ändern."

4.5 Verständnisfragen – so lernst du nebenbei

Du lernst schneller, wenn du nicht nur reparieren, sondern **verstehen** lässt:

Frage	Zweck
„Erkläre diese Funktion Zeile für Zeile."	Verstehen
„Was passiert, wenn die Datei leer ist?"	Randfälle erkennen
„Warum hast du X statt Y gewählt?"	Alternativen kennenlernen
„Was ist der einfachste Weg, das zu testen?"	Testen lernen
„Welche Teile dieses Programms sind riskant?"	Risikobewusstsein
„Erkläre das mit einem Alltagsvergleich."	Intuition aufbauen

4.6 Code-Reviews durch KI

Ein **Code-Review** ist eine kritische Durchsicht von Code. Die KI ist hier ein guter, unermüdlicher Gegenleser, aber kein Ersatz für Tests.

Beispiel 4.11: Review-Prompt

```
Reviewe den folgenden Code als erfahrener Linux-Entwickler. Ich bin Anfänger,
erkläre also verständlich. Prüfe in dieser Reihenfolge:
1. Fehler und Randfälle (leere Eingaben, fehlende Rechte, Sonderzeichen)
2. Sicherheit (Eingaben, Dateipfade, Befehlsausführung, Geheimnisse)
3. Gefahr von Datenverlust
4. Lesbarkeit und Aufbau
Gib pro Fund: Zeile, Problem, Schweregrad (hoch/mittel/niedrig), Korrekturvorschlag.
Ändere den Code nicht, sondern berichte nur.
```

Der Hinweis „ändere den Code nicht" ist wichtig: Erst bewertest du die Funde, dann entscheidest du, was umgesetzt wird.

Review-Ergebnisse einordnen

- **Nicht blind übernehmen.** Frage bei jedem Fund: Ist das ein echtes Problem für mein Vorhaben?
- **Schweregrad hinterfragen:** KI übertreibt manchmal Kleinigkeiten und übersieht manchmal Größeres.
- **Zweite Meinung:** Lass einen anderen Assistenten über denselben Code schauen. Was beide nennen, ist meist wichtig.

- **Linter ergänzen:** Werkzeuge wie `shellcheck` (Bash) und `ruff` oder `pyflakes` (Python) finden Fehler **deterministisch**, also ohne Raterei.

Beispiel 4.12: Statische Prüfwerkzeuge laufen lassen

```
$ shellcheck neues-projekt
$ python3 -m pip install ruff           # in einer venv
$ ruff check dupfinder.py
$ python3 -m py_compile dupfinder.py    # reine Syntaxprüfung
```

4.7 Tests als Prompt-Hilfe

Ein **Test** ist ein kleines Programm, das prüft, ob dein Code das Richtige tut. Lass dir Tests schreiben, **bevor** du Funktionen erweiterst. Dann bemerkst du sofort, wenn eine KI-Änderung etwas kaputt macht.

Beispiel 4.13: Ein einfacher Test mit unittest (Standardbibliothek)

```
import unittest
from namen import aufraeumen


class TestAufraeumen(unittest.TestCase):
    def test_leerzeichen_und_gross(self):
        self.assertEqual(aufraeumen("Urlaub 2024 (final).JPG"),
                         "urlaub_2024_final.jpg")
    def test_fuehrende_leerzeichen(self):
        self.assertEqual(aufraeumen("  Mein Foto!!.png"), "mein_foto.png")


if __name__ == "__main__":
    unittest.main()
```

Beispiel 4.14: Tests ausführen

```
$ python3 -m unittest -v
test_fuehrende_leerzeichen (test_namen.TestAufraeumen) ... ok
test_leerzeichen_und_gross (test_namen.TestAufraeumen) ... ok
Ran 2 tests in 0.001s
OK
```

Das Prompt-Muster dazu:

Beispiel 4.15: Tests vor der Änderung anfordern

```
Schreibe zuerst unittest-Tests für die Funktion aufraeumen(), die die Beispiele
oben und mindestens vier Randfälle abdecken (leerer String, nur Sonderzeichen,
Umlaute, sehr langer Name). Danach ändere die Funktion so, dass alle Tests bestehen.
```

Übungen

Übung

1. Verbessere den Prompt „Mach ein Skript, das meine Fotos sortiert“ nach den sechs Fragen aus 4.1.
2. Schreibe einen Prompt mit Beispiel-Ein- und Ausgabe (Muster 3) für ein Skript, das Dateien nach Erweiterung in Unterordner verschiebt.
3. Erzeuge mit `python3 -c "print(int('abc'))"` einen Fehler und lass ihn dir von der KI erklären. Lies den Traceback vorher selbst von unten nach oben.

4. Bitte die KI, ein eigenes Skript zu reviewen (Beispiel 4.11). Bewerte jeden Fund: berechtigt, übertrieben oder falsch?
5. Lass dir Tests für eine kleine Funktion schreiben und führe sie mit `python3 -m unittest -v` aus.

Lösungshinweis

Zu 1: Ein guter Prompt nennt Ordnerpfad, Sortierkriterium (Datum aus EXIF oder Dateidatum?), Zielstruktur, was bei Namenskonflikten passiert, dass nie gelöscht wird, und ein Beispiel. Zu 3: Der Traceback endet mit `ValueError: invalid literal for int() with base 10: 'abc'`.

Zusammenfassung

Zusammenfassung

- Gute Prompts klären Was, Wofür, Womit, Wie, Was nicht und Woran du Erfolg erkennst.
- Muster: Rückfragen erzwingen, Beispiele geben, Format vorgeben, Grenzen setzen, Selbstprüfung verlangen.
- Kontext in sinnvoller Reihenfolge liefern: Ziel, Umgebung, relevanter Code, Problem, Format.
- Fehlermeldungen vollständig weitergeben; Tracebacks von unten nach oben lesen.
- Lass dir erklären statt nur reparieren – so lernst du nebenbei.
- Code-Reviews durch KI sind nützlich, aber ergänze sie durch Linter und eigene Tests.
- Tests vor Änderungen schützen dich vor stillen Rückschritten.

Kapitel 5: Drei Praxisprojekte

Lernziele

- Du baust ein vollständiges CLI-Tool (Duplikate finden) mit Argumenten, Fehlerbehandlung und Tests.
- Du baust eine kleine Webanwendung (HTML, CSS, JavaScript) und erweiterst sie optional mit Flask.
- Du baust ein Automatisierungsskript mit Trockenlauf und zeitgesteuerter Ausführung.
- Du wendest in jedem Projekt den Workflow aus Kapitel 3 an.

Jedes Projekt folgt demselben Schema: **Ziel, Plan, Schritte mit Prompts, Test, Commit, Erweiterungen**. Die Prompts sind Vorschläge – formuliere sie gern um. Der Code sind typische Ergebnisse, die dir als Vergleich dienen; deine KI wird etwas anderes liefern, und das ist völlig in Ordnung, solange deine Tests bestehen.

Projekt 1: dupfinder – ein CLI-Tool in Python

Ziel

Ein Programm, das in einem Ordner doppelte Dateien (gleicher Inhalt) findet und anzeigt. Es **löscht nie** etwas, sondern berichtet nur.

Schritt 0: Projekt anlegen

Beispiel 5.1: Projekt anlegen

```
$ neues-projekt dupfinder          # Skript aus Beispiel 2.17
$ cd ~/vibe/projekte/dupfinder
$ mkdir -p tests
$ nano PROJEKT.md                  # Ziel und Rahmen eintragen (Vorlage: Beispiel 3.2)
```

Schritt 1 bis 2: Auflisten und Hashen

Die Schritte kennst du aus Kapitel 3 (Beispiele 3.3 und 3.7). Falls du sie dort nicht gebaut hast, hole das jetzt nach. Danach hast du eine Datei `dupfinder.py`, die Dateien rekursiv durchgeht und SHA-256-Hashes berechnet.

Gut zu wissen

Warum ein Hash? Ein Hash ist ein „digitaler Fingerabdruck“ eines Dateiinhalts. Die Wahrscheinlichkeit, dass zwei unterschiedliche Dateien zufällig denselben SHA-256-Wert ergeben (eine sogenannte **Kollision**), ist astronomisch klein – für den Alltag kannst du einen Treffer als Beleg für gleichen Inhalt behandeln. Mathematisch ausgeschlossen ist eine Kollision damit aber nicht, nur praktisch vernachlässigbar. So vergleichst du Dateien, ohne sie Byte für Byte nebeneinanderzulegen.

Schritt 3: Gruppieren und effizient werden

Prompt:

Beispiel 5.2: Prompt für das Gruppieren

```
Erweitere dupfinder.py. Gruppiere Dateien mit gleichem Hash. Optimierung:
Berechne den Hash nur für Dateien, deren Größe mit mindestens einer anderen
Datei übereinstimmt. Gib nur Gruppen mit mindestens 2 Dateien aus.
Nicht lesbare Dateien: Warnung auf stderr, aber weitermachen.
Füge Argparse hinzu mit --min-groesse (Bytes, Standard 1).
Zeige die vollständige Datei.
```

Ein typisches Ergebnis (vollständiges Programm):

Beispiel 5.3: dupfinder.py – vollständige Fassung

```
#!/usr/bin/env python3
"""dupfinder - findet doppelte Dateien (gleicher Inhalt) in einem Ordner."""
import argparse
import hashlib
import os
import sys
from collections import defaultdict
from pathlib import Path
BLOCK = 1024 * 1024  # 1 MiB pro Lesevorgang


def alle_dateien(ordner: Path):
    """Liefert alle regulären Dateien (keine Symlinks) unterhalb von ordner.

    Nutzt os.walk() mit followlinks=False statt Path.rglob(), weil rglob()
    standardmäßig auch in symlink-verlinkte Unterordner hineinschaut: Eine
    normale Datei, die nur über einen solchen Ordner erreichbar ist, würde
    sonst trotz der Einschränkung "keine Symlinks" mit aufgenommen.
    """
    for wurzel, unterordner, dateien in os.walk(ordner, followlinks=False):
        for name in dateien:
            pfad = Path(wurzel) / name
            if pfad.is_file() and not pfad.is_symlink():
                yield pfad


def sha256_von(datei: Path) -> str:
    h = hashlib.sha256()
    with datei.open("rb") as f:
        for block in iter(lambda: f.read(BLOCK), b""):
            h.update(block)
    return h.hexdigest()


def finde_duplikate(ordner: Path, min_groesse: int = 1):
    """Gibt eine Liste von Gruppen zurück; jede Gruppe ist eine Liste von Pfaden."""
    nach_groesse = defaultdict(list)
    for datei in alle_dateien(ordner):
        try:
            groesse = datei.stat().st_size
        except OSError as fehler:
            print(f"Warnung: {datei}: {fehler}", file=sys.stderr)
            continue
        if groesse >= min_groesse:
            nach_groesse[groesse].append(datei)

    nach_hash = defaultdict(list)
    for groesse, dateien in nach_groesse.items():
        if len(dateien) < 2:
            continue  # einzigartige Größe -> kann kein Duplikat sein
        for datei in dateien:
            try:
                nach_hash[(groesse, sha256_von(datei))].append(datei)
            except OSError as fehler:
                print(f"Warnung: {datei}: {fehler}", file=sys.stderr)
    return [sorted(gruppe) for gruppe in nach_hash.values() if len(gruppe) > 1]


def menschlich(bytes_: int) -> str:
    wert = float(bytes_)
    for einheit in ("B", "KiB", "MiB", "GiB"):
        if wert < 1024:
```

```
            return f"{wert:.1f} {einheit}"
        wert /= 1024
    return f"{wert:.1f} TiB"


def main() -> int:
    parser = argparse.ArgumentParser(description="Findet doppelte Dateien.")
    parser.add_argument("ordner", type=Path, help="zu durchsuchender Ordner")
    parser.add_argument("--min-groesse", type=int, default=1,
                        help="Mindestgröße in Bytes (Standard: 1)")
    args = parser.parse_args()

    if not args.ordner.is_dir():
        print(f"Fehler: '{args.ordner}' ist kein Ordner.", file=sys.stderr)
        return 1
    if args.min_groesse < 0:
        print("Fehler: --min-groesse darf nicht negativ sein.", file=sys.stderr)
        return 1

    gruppen = finde_duplikate(args.ordner, args.min_groesse)
    verschwendet = 0
    for nr, gruppe in enumerate(gruppen, start=1):
        groesse = gruppe[0].stat().st_size
        verschwendet += groesse * (len(gruppe) - 1)
        print(f"Gruppe {nr}: {len(gruppe)} Dateien, je {menschlich(groesse)}")
        for pfad in gruppe:
            print(f"  {pfad}")
    print(f"\n{len(gruppen)} Gruppen, vermeidbarer Platz: {menschlich(verschwendet)}")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

Schritt 4: Testen mit echten Testdaten

Teste **nie** zuerst an deinen echten Daten. Erzeuge eine kleine Testumgebung:

Beispiel 5.4: Testordner mit bekannten Duplikaten erzeugen

```
$ mkdir -p /tmp/dup-test/unter
$ echo "gleicher Inhalt" > /tmp/dup-test/a.txt
$ echo "gleicher Inhalt" > /tmp/dup-test/unter/b.txt
$ echo "anderer Inhalt"  > /tmp/dup-test/c.txt
$ python3 dupfinder.py /tmp/dup-test
Gruppe 1: 2 Dateien, je 16.0 B
  /tmp/dup-test/a.txt
  /tmp/dup-test/unter/b.txt
1 Gruppen, vermeidbarer Platz: 16.0 B
```

Das Ergebnis stimmt: a.txt und b.txt sind Duplikate, c.txt nicht. Jetzt noch ein automatischer Test, damit das auch nach späteren Änderungen so bleibt:

Beispiel 5.5: tests/test_dupfinder.py

```
import sys
import tempfile
import unittest
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
from dupfinder import finde_duplikate  # noqa: E402
class TestDuplikate(unittest.TestCase):
    def test_findet_duplikate(self):
        with tempfile.TemporaryDirectory() as tmp:
            p = Path(tmp)
            (p / "a.txt").write_text("gleich")
            (p / "b.txt").write_text("gleich")
            (p / "c.txt").write_text("anders")
            gruppen = finde_duplikate(p)
            self.assertEqual(len(gruppen), 1)
            self.assertEqual({x.name for x in gruppen[0]}, {"a.txt", "b.txt"})

    def test_leerer_ordner(self):
        with tempfile.TemporaryDirectory() as tmp:
            self.assertEqual(finde_duplikate(Path(tmp)), [])
    def test_min_groesse(self):
        with tempfile.TemporaryDirectory() as tmp:
            p = Path(tmp)
            (p / "a.txt").write_text("xx")
            (p / "b.txt").write_text("xx")
            self.assertEqual(finde_duplikate(p, min_groesse=100), [])

    def test_symlink_auf_datei_wird_uebersprungen(self):
        with tempfile.TemporaryDirectory() as tmp:
            p = Path(tmp)
            (p / "original.txt").write_text("gleich")
            (p / "verweis.txt").symlink_to(p / "original.txt")
            (p / "kopie.txt").write_text("gleich")
            gruppen = finde_duplikate(p)
            # Nur die zwei echten Dateien bilden ein Duplikat-Paar; der
            # Symlink zählt nicht mit, auch wenn er auf dieselben Bytes zeigt.
            self.assertEqual(len(gruppen), 1)
            self.assertEqual({x.name for x in gruppen[0]}, {"original.txt", "kopie.txt"})

    def test_symlink_auf_ordner_wird_nicht_verfolgt(self):
        with tempfile.TemporaryDirectory() as tmp:
            p = Path(tmp)
            echter_ordner = p / "echt"
            echter_ordner.mkdir()
            (echter_ordner / "a.txt").write_text("gleich")
            (echter_ordner / "b.txt").write_text("gleich")
            (p / "verweis_ordner").symlink_to(echter_ordner)
            # Ohne den Symlink zu verfolgen, wird jede Datei nur einmal
            # gefunden (über "echt/"), nicht zusätzlich über "verweis_ordner/".
            gruppen = finde_duplikate(p)
            alle_namen = [x.name for gruppe in gruppen for x in gruppe]
            self.assertEqual(sorted(alle_namen), ["a.txt", "b.txt"])
if __name__ == "__main__":
    unittest.main()
```

Beispiel 5.6: Tests ausführen und committen

```
$ python3 -m unittest discover -s tests -v
...
Ran 3 tests in 0.004s
OK
$ git add -A && git commit -m "dupfinder: Gruppieren, argparse, Tests"
```

Schritt 5: Komfortabel aufrufen

Beispiel 5.7: dupfinder wie ein echtes Linux-Programm installieren

```
$ mkdir -p ~/.local/bin                        # falls noch nicht vorhanden
$ chmod +x dupfinder.py
$ ln -s "$PWD/dupfinder.py" ~/.local/bin/dupfinder
$ echo "$PATH" | tr ':' '\n' | grep -q "$HOME/.local/bin" \
    && echo "im PATH" || echo "NICHT im PATH"
$ dupfinder ~/Downloads --min-groesse 1048576
```

> **Gut zu wissen**
>
> Der Symlink zeigt über `$PWD` auf den **aktuellen Pfad** der Datei. Verschiebst du das Projekt später an einen anderen Ort, zeigt `~/.local/bin/dupfinder` ins Leere. Mit `ls -l ~/.local/bin/dupfinder` siehst du das Ziel des Symlinks; lege ihn nach einem Umzug einfach neu an. Fehlt `~/.local/bin` noch im `PATH`, trägst du es wie in Beispiel 2.18 beschrieben ein.

Erweiterungen zum Üben

- Ausgabe als JSON mit `--json` (nützlich für andere Programme, `jq` kann sie auswerten).
- Option `--ignoriere ".git"`, um bestimmte Ordner zu überspringen.
- **Sicher** löschen: Duplikate in einen Papierkorb-Ordner **verschieben**, nicht löschen, und zwar nur mit ausdrücklicher Bestätigung.

> **WARNUNG – Sicherheitsrisiko**
>
> Ein Programm, das Dateien löscht, ist der klassische Ort für Datenverlust. Wenn du die Löschfunktion baust: Standard ist immer der **Trockenlauf** (nur anzeigen), echtes Handeln nur mit einer Option wie `--ausfuehren`, und teste das zuerst in `/tmp`.

Projekt 2: Ein kleines Web-Projekt

Ziel

Eine Aufgabenliste im Browser: Aufgaben hinzufügen, abhaken, löschen, Daten bleiben nach dem Neuladen erhalten. **Eine einzige Datei**, ohne Server und ohne Installation.

Plan

1. Seite mit Eingabefeld und Liste (HTML und CSS)
2. Aufgaben hinzufügen (JavaScript)
3. Abhaken und Löschen
4. Speichern im Browser (`localStorage`)

Wenn du das HTML-Grundgerüst schon aus früheren Unterlagen kennst: Hier zählt nur, wie die Teile zusammenspielen. **HTML** liefert den Inhalt, **CSS** das Aussehen, **JavaScript** das Verhalten.

Prompt

Beispiel 5.8: Prompt für die Aufgabenliste

```
Baue eine Aufgabenliste als einzelne Datei index.html (HTML, CSS und JavaScript
in einer Datei, keine externen Bibliotheken, kein Netzwerkzugriff).
Funktionen: Aufgabe eingeben und mit Enter oder Button hinzufügen; Aufgabe
per Klick abhaken (durchgestrichen); Löschen-Button pro Aufgabe; Speichern
im localStorage. Leere Eingaben ignorieren. Oberfläche auf Deutsch, schlicht
und gut lesbar, auch auf dem Smartphone. Erkläre am Ende kurz den Aufbau.
```

Typisches Ergebnis:

Beispiel 5.9: index.html – Aufgabenliste (eine Datei)

```
<!DOCTYPE html>
<html lang="de">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Aufgabenliste</title>
  <style>
    body { font-family: system-ui, sans-serif; max-width: 32rem; margin: 2rem auto;
           padding: 0 1rem; color: #1f2933; }
    h1 { color: #0b6e4f; }
    form { display: flex; gap: .5rem; margin-bottom: 1rem; }
    input { flex: 1; padding: .6rem; font-size: 1rem; }
    button { padding: .6rem 1rem; font-size: 1rem; cursor: pointer; }
    ul { list-style: none; padding: 0; }
    li { display: flex; align-items: center; gap: .5rem; padding: .5rem 0;
         border-bottom: 1px solid #dde3e8; }
    li span { flex: 1; cursor: pointer; }
    li.fertig span { text-decoration: line-through; color: #8a949e; }
  </style>
</head>
<body>
  <h1>Meine Aufgaben</h1>
  <form id="formular">
    <input id="eingabe" placeholder="Neue Aufgabe ..." autocomplete="off">
    <button type="submit">Hinzufügen</button>
  </form>
  <ul id="liste"></ul>

  <script>
    const SCHLUESSEL = "aufgaben-v1";
    let aufgaben = [];
    try {
      aufgaben = JSON.parse(localStorage.getItem(SCHLUESSEL)) || [];
    } catch (e) {
      aufgaben = [];
    }

    const liste = document.getElementById("liste");
    const formular = document.getElementById("formular");
    const eingabe = document.getElementById("eingabe");
    function speichern() {
      localStorage.setItem(SCHLUESSEL, JSON.stringify(aufgaben));
    }

    function zeichnen() {
      liste.innerHTML = "";
      aufgaben.forEach((a, index) => {
        const li = document.createElement("li");
        if (a.fertig) li.className = "fertig";
        const text = document.createElement("span");
        text.textContent = a.text;           // textContent: sicher gegen Code-Einschleusung
        text.onclick = () => { a.fertig = !a.fertig; speichern(); zeichnen(); };
        const loeschen = document.createElement("button");
        loeschen.textContent = "Löschen";
        loeschen.onclick = () => { aufgaben.splice(index, 1); speichern(); zeichnen(); };
        li.append(text, loeschen);
        liste.append(li);
      });
    }

    formular.addEventListener("submit", (ereignis) => {
      ereignis.preventDefault();
      const text = eingabe.value.trim();
```

```
      if (!text) return;
      aufgaben.push({ text: text, fertig: false });
      eingabe.value = "";
      speichern();
      zeichnen();
    });

    zeichnen();
  </script>
</body>
</html>
```

Ausprobieren

Du kannst die Datei direkt im Browser öffnen oder einen lokalen Mini-Server starten:

Beispiel 5.10: Seite lokal ansehen

```
$ cd ~/vibe/projekte/aufgabenliste
$ xdg-open index.html                          # öffnet die Datei im Standardbrowser
$ python3 -m http.server 8000 --bind 127.0.0.1  # alternativ: Server, nur lokal erreichbar
# Browser: http://localhost:8000
```

`--bind 127.0.0.1` ist hier bewusst von Anfang an dabei: Ohne diese Option lauscht der Server standardmäßig auf **allen** Netzwerkschnittstellen, ist also auch für andere Geräte im selben Netz erreichbar – in einem vertrauten Heimnetz meist unkritisch, in einem fremden Netz (Café, Hotel, Konferenz) aber ein unnötiges Risiko für eine absichtlich unsichere Übungsanwendung wie diese.

Beispiel 5.11: Zum Vergleich – der ungefilterte Aufruf (bewusst, mit Erklärung)

```
$ python3 -m http.server 8000
# Jetzt auch von anderen Geräten im selben Netz erreichbar, z. B.
# http://<deine-lokale-IP>:8000 – nützlich zum Testen vom Smartphone aus,
# aber nur in Netzen, denen du vertraust.
```

> **TIPP – Profi-Trick**
>
> Öffne im Browser die **Entwicklerwerkzeuge** mit `F12`. Im Reiter „Konsole“ siehst du JavaScript-Fehler, im Reiter „Anwendung“ (oder „Speicher“) siehst du den `localStorage`. Kopiere Fehlermeldungen aus der Konsole in deine Anfrage an die KI.

Teste wie ein Nutzer

Probiere bewusst Unfug aus: leere Eingabe, ein sehr langer Text, Sonderzeichen wie `<b>fett</b>` (es muss als Text erscheinen, nicht als Formatierung), Seite neu laden. Das Letzte ist ein klassischer Sicherheitstest: Würde das Programm den Text als HTML interpretieren, könnte man Schadcode einschleusen (**XSS**, siehe Kapitel 6). Deshalb nutzt der Code `textContent` statt `innerHTML` für Nutzertexte.

Optional: Mit Flask zu einem echten Server

Wenn Daten nicht nur im Browser, sondern auf einem Server liegen sollen, brauchst du ein **Backend**. **Flask** ist ein kleines Python-Framework dafür.

Beispiel 5.12: Flask installieren

```
$ cd ~/vibe/projekte/aufgabenliste
$ python3 -m venv .venv && source .venv/bin/activate
(.venv) $ pip install flask
```

Beispiel 5.13: app.py – minimaler Flask-Server mit JSON-Schnittstelle

```
from flask import Flask, jsonify, request, send_from_directory

app = Flask(__name__)
aufgaben = []          # nur im Speicher: nach Neustart weg (für den Anfang okay)


@app.get("/")
def startseite():
    return send_from_directory(".", "index.html")


@app.get("/api/aufgaben")
def liste():
    return jsonify(aufgaben)
@app.post("/api/aufgaben")
def neu():
    daten = request.get_json(silent=True) or {}
    text = str(daten.get("text", "")).strip()
    if not text or len(text) > 200:
        return jsonify(fehler="Text fehlt oder ist zu lang"), 400
    aufgaben.append({"text": text, "fertig": False})
    return jsonify(aufgaben[-1]), 201
if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, debug=False)
```

Beispiel 5.14: Server starten und mit curl testen

```
(.venv) $ python3 app.py &
(.venv) $ curl -s -X POST localhost:5000/api/aufgaben \
    -H 'Content-Type: application/json' -d '{"text":"Milch kaufen"}'
{"fertig":false,"text":"Milch kaufen"}
(.venv) $ curl -s localhost:5000/api/aufgaben | jq .
(.venv) $ kill %1
```

WARNUNG – Sicherheitsrisiko

Starte Flask niemals mit `debug=True` auf einem Rechner, der aus dem Netz erreichbar ist: Der Debugger erlaubt das Ausführen von Code. Für den echten Betrieb im Internet brauchst du einen Produktionsserver (zum Beispiel gunicorn hinter einem Reverse-Proxy) und weitere Absicherung – das ist ein eigenes Thema.

Projekt 3: Ein Automatisierungsskript

Ziel

Ein Skript, das deinen Ordner `~/Downloads` aufräumt: Dateien wandern nach Typ in Unterordner (`Bilder`, `Dokumente`, `Archive`, `Sonstiges`). Es hat einen **Trockenlauf** als Standard und läuft auf Wunsch automatisch jeden Tag.

Plan

1. Dateien nach Endung einer Kategorie zuordnen
2. Trockenlauf: nur anzeigen, was passieren würde
3. Echtes Verschieben, ohne vorhandene Dateien zu überschreiben
4. Protokoll
5. Zeitsteuerung

Prompt

Beispiel 5.15: Prompt für das Aufräumskript

```
Schreibe aufraeumen.py (Python 3, nur Standardbibliothek) für Linux.
Es räumt einen Ordner auf (Argument, Standard ~/Downloads), indem es Dateien
direkt in diesem Ordner nach Endung in Unterordner verschiebt:
Bilder (jpg jpeg png gif webp svg), Dokumente (pdf txt odt docx md csv),
Archive (zip tar gz xz 7z), Sonstiges (alles andere).
Wichtig: Standard ist ein Trockenlauf, der nur ausgibt, was passieren würde.
Erst mit --ausfuehren wird verschoben. Nie etwas überschreiben: Bei Namens-
konflikt "name (2).endung" verwenden. Unterordner und versteckte Dateien
(Name beginnt mit Punkt) nicht anfassen. Jede Aktion in ein Logfile schreiben.
```

Typisches Ergebnis:

Beispiel 5.16: aufraeumen.py

```
#!/usr/bin/env python3
"""aufraeumen - sortiert Dateien eines Ordners nach Typ in Unterordner."""
import argparse
import logging
import os
import shutil
import sys
from pathlib import Path
KATEGORIEN = {
    "Bilder": {"jpg", "jpeg", "png", "gif", "webp", "svg"},
    "Dokumente": {"pdf", "txt", "odt", "docx", "md", "csv"},
    "Archive": {"zip", "tar", "gz", "xz", "7z"},
}
def kategorie_von(datei: Path) -> str:
    endung = datei.suffix.lower().lstrip(".")
    for name, endungen in KATEGORIEN.items():
        if endung in endungen:
            return name
    return "Sonstiges"
def freier_name(ziel: Path) -> Path:
    """Reserviert atomar einen freien Namen; hängt bei Bedarf (2), (3) ... an.

    Reserviert heißt: Die Funktion legt am zurückgegebenen Pfad bereits eine
    leere Platzhalterdatei an, bevor sie zurückkehrt. Das schließt die Lücke
    zwischen "ist der Name frei?" und "jetzt verschieben" (siehe Warnbox
    unten): Zwischen beiden Schritten kann sonst eine andere Datei genau
    diesen Namen belegen. os.O_EXCL sorgt zusätzlich dafür, dass auch ein
    bereits vorhandener, defekter Symlink als belegt erkannt wird, obwohl
    Path.exists() für einen solchen Symlink fälschlich False liefert.
    """
    kandidat = ziel
    zaehler = 2
    while True:
        try:
            fd = os.open(kandidat, os.O_CREAT | os.O_EXCL | os.O_WRONLY)
            os.close(fd)
            return kandidat
        except FileExistsError:
            kandidat = ziel.with_name(f"{ziel.stem} ({zaehler}){ziel.suffix}")
            zaehler += 1


def main() -> int:
    parser = argparse.ArgumentParser(description="Ordner nach Dateityp aufräumen.")
    parser.add_argument("ordner", nargs="?", type=Path,
                        default=Path.home() / "Downloads")
    parser.add_argument("--ausfuehren", action="store_true",
                        help="wirklich verschieben (Standard: nur anzeigen)")
    args = parser.parse_args()

    if not args.ordner.is_dir():
        print(f"Fehler: '{args.ordner}' ist kein Ordner.", file=sys.stderr)
        return 1

    log_pfad = Path.home() / ".local" / "state" / "aufraeumen.log"
    log_pfad.parent.mkdir(parents=True, exist_ok=True)
    logging.basicConfig(filename=log_pfad, level=logging.INFO,
                        format="%(asctime)s %(message)s")
    anzahl = 0
    for datei in sorted(args.ordner.iterdir()):
        if not datei.is_file() or datei.name.startswith("."):
            continue
        ziel_ordner = args.ordner / kategorie_von(datei)
        if args.ausfuehren:
            ziel_ordner.mkdir(exist_ok=True)
            # freier_name() reserviert den Namen atomar (siehe Kommentar dort).
```

```
            ziel = freier_name(ziel_ordner / datei.name)
            shutil.move(str(datei), str(ziel))  # überschreibt die eigene Platzhalterdatei
            logging.info("verschoben: %s -> %s", datei, ziel)
        else:
            # Im Trockenlauf nur den voraussichtlichen Namen anzeigen, ohne
            # eine Platzhalterdatei anzulegen oder den Zielordner zu erstellen.
            vorschau = ziel_ordner / datei.name
            zaehler = 2
            while vorschau.exists() or os.path.lexists(vorschau):
                vorschau = vorschau.with_name(f"{datei.stem} ({zaehler}){datei.suffix}")
                zaehler += 1
            ziel = vorschau
        print(f"{'VERSCHOBEN' if args.ausfuehren else 'würde verschieben'}: "
              f"{datei.name} -> {ziel_ordner.name}/{ziel.name}")
        anzahl += 1
    if not args.ausfuehren:
        print(f"\nTrockenlauf: {anzahl} Dateien. Mit --ausfuehren wirklich verschieben.")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

> **Gut zu wissen**
>
> **Warum die Platzhalterdatei?** Zwischen „der Name ist frei" und „jetzt verschieben" kann eine andere Datei genau diesen Namen belegen (ein sogenanntes Zeitfenster- oder Race-Condition-Problem). `os.open(..., os.O_CREAT | os.O_EXCL)` legt den Namen **in einem einzigen, nicht unterbrechbaren Schritt** als leere Datei an und schlägt fehl, wenn er bereits existiert – auch wenn es sich um einen **defekten Symlink** handelt, den `Path.exists()` fälschlich als „nicht vorhanden" melden würde. Erst danach verschiebt `shutil.move()` die echte Datei dorthin und überschreibt dabei gezielt die eigene, gerade erst angelegte Platzhalterdatei. Im Trockenlauf entfällt diese Reservierung bewusst: Es wird nichts angelegt oder verändert, die Namensvorschau kann sich deshalb theoretisch bis zum echten Lauf noch ändern.

Sicher testen

Beispiel 5.17: Test im Wegwerf-Ordner statt in den echten Downloads

```
$ mkdir /tmp/dl-test && cd /tmp/dl-test
$ touch foto.jpg foto.JPG bericht.pdf archiv.tar.gz notiz.xyz .versteckt
$ python3 ~/vibe/projekte/aufraeumen/aufraeumen.py /tmp/dl-test
würde verschieben: archiv.tar.gz -> Archive/archiv.tar.gz
würde verschieben: bericht.pdf -> Dokumente/bericht.pdf
...
$ python3 ~/vibe/projekte/aufraeumen/aufraeumen.py /tmp/dl-test --ausfuehren
$ tree /tmp/dl-test
```

Prüfe: Wurde `.versteckt` in Ruhe gelassen? Landete `foto.JPG` bei den Bildern? Haben beide Fotos überlebt (nichts überschrieben)? Erst wenn alles stimmt, probiere es an einer **Kopie** deiner echten Downloads.

Zeitsteuerung mit systemd-Timer

Unter modernen Linux-Systemen laufen Benutzer-Timer ohne Root-Rechte. Du legst zwei Dateien an:

Wichtig: Richte den Timer zuerst **ohne** `--ausfuehren` ein und lass ihn einige Tage als reinen Trockenlauf mitlaufen, bevor du ihn scharf schaltest. So steht es weiter unten in der Warnbox – deshalb beginnt auch der folgende Service bewusst ohne diese Option:

Beispiel 5.18: ~/.config/systemd/user/aufraeumen.service (Trockenlauf-Fassung)

```
[Unit]
Description=Downloads aufräumen (Trockenlauf)

[Service]
Type=oneshot
ExecStart=%h/.local/bin/aufraeumen
```

Beispiel 5.19: ~/.config/systemd/user/aufraeumen.timer

```
[Unit]
Description=Downloads täglich aufräumen

[Timer]
OnCalendar=*-*-* 18:00:00
Persistent=true

[Install]
WantedBy=timers.target
```

Beispiel 5.20: Timer einrichten, OHNE ihn sofort zu starten

```
$ mkdir -p ~/.config/systemd/user
$ ln -s ~/vibe/projekte/aufraeumen/aufraeumen.py ~/.local/bin/aufraeumen
$ chmod +x ~/vibe/projekte/aufraeumen/aufraeumen.py
$ systemctl --user daemon-reload
$ systemctl --user enable aufraeumen.timer   # OHNE --now: aktiviert, startet nicht sofort
$ systemctl --user start aufraeumen.timer    # startet den Timer selbst (nicht den ersten Lauf)
$ systemctl --user list-timers | grep aufraeumen
$ journalctl --user -u aufraeumen.service -n 20
```

`%h` steht in systemd-Dateien für dein Home-Verzeichnis. `Persistent=true` holt verpasste Läufe nach, wenn der Rechner um 18 Uhr aus war – das gilt später auch für den Trockenlauf, du bekommst also auch nach einer Downtime eine Vorschau zu sehen, ohne dass dabei etwas verschoben wird.

> **Gut zu wissen**
>
> Die klassische Alternative ist **cron**: `crontab -e` und eine Zeile wie `0 18 * $HOME/.local/bin/aufraeumen`. Beide Wege sind gut; cron ist einfacher, systemd-Timer bieten bessere Protokollierung.

> **WARNUNG – Sicherheitsrisiko**
>
> Ein **automatisch** laufendes Skript, das Dateien verschiebt, ist riskanter als ein von Hand gestartetes. Lass den Timer in der Trockenlauf-Fassung oben zuerst einige Tage laufen und lies das Ergebnis im Journal (`journalctl --user -u aufraeumen.service`). **Erst danach** trägst du `--ausfuehren` in die `ExecStart`-Zeile des Service ein, wie in Beispiel 5.20b gezeigt.

Beispiel 5.20b: Erst nach kontrollierter Sichtung auf echten Betrieb umstellen

```
$ systemctl --user stop aufraeumen.timer
$ nano ~/.config/systemd/user/aufraeumen.service   # ExecStart=...aufraeumen --ausfuehren
$ systemctl --user daemon-reload
$ systemctl --user start aufraeumen.timer
```

Den Timer wieder abschalten geht jederzeit mit:

Beispiel 5.20c: Timer deaktivieren

```
$ systemctl --user disable --now aufraeumen.timer
```

Übungen

Übung

1. Erweitere `dupfinder` um `--json`: Die Ausgabe soll eine JSON-Liste von Gruppen sein. Prüfe sie mit `| jq .`
2. Ändere die Aufgabenliste so, dass jede Aufgabe ein Datum bekommt. Lass die KI nur die nötigen Änderungen zeigen und prüfe sie mit `git diff`.
3. Gib `aufraeumen.py` eine Option `--rueckgaengig`, die anhand des Logfiles die letzten Verschiebungen umkehrt. Entwirf zuerst gemeinsam mit der KI den Plan (Kapitel 3.2) und teste im Wegwerf-Ordner.
4. Baue die Flask-Anwendung so aus, dass Aufgaben in einer Datei `aufgaben.json` gespeichert werden und einen Neustart überleben.
5. Lass die KI alle drei Projekte nach Beispiel 4.11 reviewen und bewerte die Funde.

Lösungshinweis

Zu 3: Das bisherige Logformat „verschoben: Quelle -> Ziel“ ist als Freitext gedacht und für eine automatische Rückgängig-Funktion ungeeignet, sobald ein Pfad selbst ein `->` enthalten könnte. Für `--rueckgaengig` lohnt sich ein **strukturiertes** Logformat, zum Beispiel eine Zeile JSON pro Vorgang (JSONL) mit getrennten Feldern für Quelle und Ziel, etwa `{"quelle": "...", "ziel": "..."}`. So lässt sich jede Zeile eindeutig einlesen, unabhängig davon, welche Zeichen im Dateinamen vorkommen. Prüfe beim Rückgängigmachen zusätzlich, ob die ursprüngliche Quelle inzwischen durch eine andere Datei belegt ist, und biete auch dafür zuerst einen Trockenlauf an. Zu 4: Beim Speichern zuerst in eine temporäre Datei schreiben und dann umbenennen, damit ein Absturz die Datei nicht halb geschrieben zurücklässt.

Zusammenfassung

Zusammenfassung

- Alle drei Projekte folgten demselben Weg: Plan, kleine Schritte, Test im Wegwerf-Umfeld, Commit.
- Tools, die Dateien verändern, bekommen immer einen Trockenlauf als Standard.
- Web-Projekte testest du wie ein misstrauischer Nutzer: leere, lange und seltsame Eingaben.
- Nutzertexte gehören mit `textContent` statt `innerHTML` in die Seite.
- Zeitgesteuerte Skripte erst als Trockenlauf beobachten, dann scharf schalten.
- Eigene Tests (`unittest`) halten das Verhalten stabil, wenn die KI später etwas ändert.

Kapitel 6: Sicherheit und Qualität

Lernziele

- Du gehst mit API-Keys und Zugangsdaten sicher um und erkennst Leaks.
- Du prüfst KI-Code systematisch auf typische Sicherheitsprobleme.
- Du kennst die Grundlagen zu Lizenzen und Datenschutz.
- Du weißt, wann du KI-Code nicht vertrauen darfst.

6.1 Warum Sicherheit hier besonders wichtig ist

Beim Vibe Coding entsteht viel Code schnell, und du prüfst ihn weniger genau, als wenn du ihn selbst geschrieben hättest. Das ist das Kernrisiko. Studien und Erfahrungsberichte zeigen immer wieder, dass KI-generierter Code Schwachstellen enthalten kann, weil Modelle Muster aus öffentlichem Code übernehmen, darunter auch unsichere. Das heißt nicht, dass du es lassen sollst, sondern dass du **eine Prüfroutine** brauchst.

Die drei größten Risikoquellen für Einsteiger:

1. **Geheimnisse an falscher Stelle** (Keys im Code, im Chat oder im Repository)
2. **Ungeprüfte Befehlsausführung** (KI-Befehle mit `sudo`, `rm -rf`, `curl | bash`)
3. **Unsicherer Code, der „funktioniert“** (fehlende Eingabeprüfung, offene Server)

6.2 API-Keys und Geheimnisse

Die goldenen Regeln

Regel	Warum
Keys nie im Quellcode	Code wird kopiert, geteilt, committet
Keys nie in Chats einfügen	Gesprächsinhalte können gespeichert werden
Keys nie committen	Git vergisst nichts, auch nach dem Löschen
Je Projekt eigener Key	Bei Verlust drehst du nur einen ab
Ausgabenlimit setzen	Begrenzt den Schaden bei Missbrauch
Rechte minimal	Nur die nötigen Berechtigungen vergeben

Keys in Programmen verwenden

Das Programm liest den Key aus der **Umgebungsvariable**, die du wie in Beispiel 2.12 bereitgestellt hast:

Beispiel 6.1: Key aus der Umgebung lesen, nie im Code speichern

```
import os
import sys

key = os.environ.get("ANTHROPIC_API_KEY")
if not key:
    print("Fehler: ANTHROPIC_API_KEY ist nicht gesetzt.", file=sys.stderr)
    sys.exit(1)
```

Das Programm gibt den Key niemals aus und schreibt ihn nicht ins Log. Prüfe das nach, wenn die KI Diagnose-Ausgaben eingebaut hat.

Leck-Suche vor jedem Commit

Beispiel 6.2: Nach Geheimnissen im Projekt suchen

```
$ rg -n -i "(api[_-]?key|secret|token|passwort|password)\s*[=:]" --glob '!.venv'
$ git diff --cached | rg -i "key|token|secret"
$ git status --short | rg "\.env"
```

Wenn Treffer erscheinen, prüfe jeden einzelnen. Es gibt auch spezialisierte Werkzeuge, die Repositories nach Geheimnissen durchsuchen (zum Beispiel `gitleaks` oder `trufflehog`). Frage deine KI nach aktuellen Empfehlungen und prüfe die Projektseite.

Was tun, wenn ein Key geleakt ist?

WARNUNG – Sicherheitsrisiko

Sofortmaßnahmen bei einem geleakten Key:

1. **Key im Anbieter-Portal sofort widerrufen** und einen neuen erzeugen. Das ist Schritt 1, vor allem anderen.
2. Nutzung und Kosten im Konto prüfen, Auffälligkeiten melden.
3. Key aus Code und Historie entfernen. Das Löschen in einem neuen Commit **reicht nicht**, denn die Historie enthält den Key weiter. Das Umschreiben der Historie ist nachrangig, weil der alte Key ohnehin ungültig sein muss.
4. Ursache beheben (`.gitignore`, Umgebungsvariable) und künftig vor jedem Commit prüfen.

6.3 Gefährliche Befehle erkennen

Terminal-Agenten und Chat-Antworten schlagen Befehle vor. **Du bist der Sicherheitsmechanismus.** Lerne diese Warnzeichen:

Muster	Gefahr
`rm -rf` mit Variablen oder `*`	kann ganze Verzeichnisse oder das System löschen
`sudo` bei unbekannten Befehlen	volle Systemrechte
`curl ...` direkt an `bash` oder `sudo bash` weitergereicht	führt Fremdcode ungeprüft aus
`chmod -R 777`	macht alles für alle schreibbar
`dd`, `mkfs`, `> /dev/...`	kann Datenträger überschreiben
`git push --force`, `git reset --hard`	zerstört Historie oder lokale Arbeit
`pip install` unbekannter Pakete	kann Schadpakete enthalten (siehe 6.6)
Änderungen an `~/.ssh`, `/etc`, `~/.bashrc`	verändert Sicherheit oder Systemverhalten

Die Drei-Fragen-Prüfung vor jedem Befehl

1. **Verstehe ich, was der Befehl tut?** Wenn nein: Lass ihn dir Teil für Teil erklären.
2. **Was ist das Schlimmste, das passieren kann?** Wenn die Antwort „Daten weg“ lautet: Backup oder Test in `/tmp`.

3. **Kann ich es zuerst trocken ausprobieren?** Viele Programme haben dafür Optionen.

Beispiel 6.3: Trockenläufe und sichere Gewohnheiten

```
$ rsync -av --dry-run quelle/ ziel/        # zeigt nur, was kopiert würde
$ rm -i datei.txt                          # fragt vor dem Löschen
$ ls ordner/*.tmp                          # erst ansehen, was der Platzhalter trifft ...
$ rm ordner/*.tmp                          # ... dann löschen
$ explainshell                             # Idee: Befehle erklären lassen (Webdienst)
```

> **TIPP – Profi-Trick**
>
> Bitte die KI bei jedem riskanten Befehl: „Erkläre jeden Teil dieses Befehls und nenne das schlimmste mögliche Ergebnis.“ Und: Starte Agenten nur in Projektordnern, nicht im Home-Verzeichnis (siehe 2.5).

Isolation: Wegwerf-Umgebungen

Wenn du etwas Riskantes ausprobieren möchtest, tue es in einer **Wegwerf-Umgebung**: einer virtuellen Maschine, einem Container oder einem eigenen Benutzerkonto ohne wichtige Daten. Container-Werkzeuge wie Docker oder Podman und Virtualisierung mit `virt-manager` sind dafür verbreitet. Ein eigener Benutzer ist der einfachste Anfang:

Beispiel 6.4: Eigenen Testbenutzer anlegen (Debian/Ubuntu)

```
$ sudo adduser vibetest
$ su - vibetest          # in das Testkonto wechseln
$ exit                   # wieder zurück
```

6.4 Typische Schwachstellen in KI-Code

Du musst kein Sicherheitsexperte werden, aber diese Muster solltest du kennen. Lass die KI gezielt danach suchen (Beispiel 4.11, Punkt 2).

Befehlseinschleusung (Command Injection)

Wenn Nutzereingaben in einen Shell-Befehl einfließen, kann ein Angreifer eigene Befehle anhängen.

Beispiel 6.5: Unsicher und sicher – Befehle aus Eingaben bauen

```
import subprocess

dateiname = input("Datei: ")      # Testeingabe: x; echo GEHACKT

# UNSICHER: Die Eingabe wird von der Shell interpretiert!
ergebnis = subprocess.run(f"cat {dateiname}", shell=True,
                          capture_output=True, text=True)
print("Ausgabe:", ergebnis.stdout, ergebnis.stderr)
# SICHER: Argumente als Liste, keine Shell
ergebnis = subprocess.run(["cat", "--", dateiname],
                          capture_output=True, text=True)
print("Ausgabe:", ergebnis.stdout, ergebnis.stderr)
```

Beispiel 6.5b: Erwartetes Ergebnis beider Varianten (Testeingabe: x; echo GEHACKT)

```
Unsicher (shell=True):
  Ausgabe: cat: x: Datei oder Verzeichnis nicht gefunden
           GEHACKT
  -> Die Shell hat den Strichpunkt als Befehlstrenner gelesen und "echo GEHACKT"
     als EIGENEN, zweiten Befehl ausgeführt.

Sicher (Liste statt shell=True):
  Ausgabe: cat: x; echo GEHACKT: Datei oder Verzeichnis nicht gefunden
  -> Die gesamte Eingabe wurde als EIN Dateiname an cat übergeben und nicht
     als Befehl interpretiert; "echo GEHACKT" wurde nicht ausgeführt.
```

Merke: `shell=True` **mit Nutzereingaben ist fast immer falsch.** Ebenso gefährlich sind `eval()` und `exec()` mit fremden Daten. Teste dieses Beispiel ausschließlich mit der harmlosen Eingabe `x; echo GEHACKT` und eigenen, dafür angelegten Testdateien – nie mit `rm` oder echten Pfaden.

Pfad-Tricks (Path Traversal)

Ein Eingabewert wie `../../etc/passwd` kann aus dem vorgesehenen Ordner ausbrechen:

Beispiel 6.6: Pfade absichern

```
from pathlib import Path

BASIS = Path("/srv/daten").resolve()

def sicherer_pfad(name: str) -> Path:
    pfad = (BASIS / name).resolve()
    if BASIS not in pfad.parents and pfad != BASIS:
        raise ValueError("Pfad außerhalb des erlaubten Ordners")
    return pfad
```

SQL-Einschleusung

Wenn du mit Datenbanken arbeitest: Baue SQL nie durch Textverkettung, sondern mit **Platzhaltern**.

Beispiel 6.7: Parametrisierte SQL-Abfrage mit sqlite3

```
import sqlite3

verbindung = sqlite3.connect("daten.db")
name = input("Name: ")

# UNSICHER: f"SELECT * FROM nutzer WHERE name = '{name}'"
zeilen = verbindung.execute("SELECT * FROM nutzer WHERE name = ?", (name,)).fetchall()
```

Cross-Site-Scripting (XSS)

Nutzertexte, die ungefiltert als HTML in eine Seite gelangen, können Schadcode ausführen. In JavaScript: `textContent` statt `innerHTML`. In Flask-Templates schützt die automatische Maskierung, solange du `|safe` nicht leichtfertig verwendest.

Weitere Dauerbrenner

- **Offene Server:** Ein Dienst auf `0.0.0.0` ist aus dem ganzen Netz erreichbar. Standard: `127.0.0.1`.
- **Schwache oder fehlende Anmeldung:** Eigene Login-Systeme sind schwer. Nutze bewährte Bibliotheken oder gar keine eigene Anmeldung.
- **Passwörter selbst speichern:** Niemals im Klartext. Wenn überhaupt, nutze bewährte Verfahren (zum Beispiel `argon2`, `bcrypt`), und frage dich zuerst, ob du das wirklich selbst bauen musst.

- **Fehlende Fehlerbehandlung:** Abstürze, die Zustände halb geändert hinterlassen.
- **Zu weite Rechte:** Dateien mit `chmod 777`, Dienste als root.

6.5 Qualitätssicherung in der Praxis

Eine Mindest-Routine, bevor du ein Programm „fertig“ nennst:

1. **Läuft es?** Normalfall, Fehlerfall und Randfälle ausprobiert.
2. **Tests** vorhanden und grün (`python3 -m unittest discover`).
3. **Linter** ohne ernste Meldungen (`ruff`, `shellcheck`).
4. `git diff` **gelesen:** Nichts Unerwartetes dabei.
5. **Geheimnis-Suche** durchgeführt (Beispiel 6.2).
6. **Abhängigkeiten geprüft:** Jedes Paket in `requirements.txt` ist bekannt und nötig.
7. **README** erklärt Zweck, Installation und Aufruf.
8. **Backup oder Git-Stand** vorhanden, bevor das Programm echte Daten anfasst.

Beispiel 6.8: Prüfroutine als kleines Skript

```
#!/usr/bin/env bash
# pruefe.sh - Mindestprüfung vor einem Commit
set -u
fehler=0
echo "== Tests =="
python3 -m unittest discover -s tests || fehler=1
echo "== Linter =="
command -v ruff >/dev/null && { ruff check . || fehler=1; }
echo "== Geheimnisse =="
if rg -n -i "(api[_-]?key|secret|token|password)\s*[=:]\s*['\"][^'\"]{8,}" \
     --glob '!.venv' --glob '!*.md' .; then
    echo "Möglicher Fund! Bitte prüfen."; fehler=1
fi
[ "$fehler" -eq 0 ] && echo "Alles in Ordnung." || echo "Es gibt Probleme."
exit "$fehler"
```

6.6 Abhängigkeiten und Lieferkette

Jedes Paket, das du installierst, ist fremder Code mit deinen Rechten. Zwei Probleme begegnen dir bei KI-Code besonders:

- **Erfundene Paketnamen:** Die KI schlägt ein Paket vor, das es nicht gibt. Angreifer registrieren solche Namen mitunter, um Schadcode zu verteilen („Slopsquatting“). Prüfe vor jeder Installation, ob das Paket real, bekannt und gepflegt ist.
- **Tippfehler-Pakete:** Namen, die bekannten Paketen ähneln (`reqeusts` statt `requests`).

Beispiel 6.9: Ein Paket vor der Installation prüfen

```
$ pip index versions requests          # existiert es, welche Versionen?
$ pip download requests --no-deps -d /tmp/pk   # nur laden, nichts installieren
$ python3 -m pip show requests         # nach der Installation: Autor, Seite, Lizenz
```

Schaue außerdem auf der Projektseite nach: Wie alt, wie verbreitet, wer pflegt es? Bevorzuge die **Standardbibliothek**, wo sie ausreicht. Weniger Abhängigkeiten bedeuten weniger Risiko. Du kannst die KI ausdrücklich dazu auffordern: „Nur Standardbibliothek“.

WARNUNG – Sicherheitsrisiko

Installiere niemals Pakete oder Skripte blind, weil „die KI es gesagt hat“. Besonders `curl ... | sudo bash`-Einzeiler sind ein Sicherheitsrisiko. Lade das Skript stattdessen herunter, lies es (oder lass es dir erklären) und führe es erst dann aus.

6.7 Lizenzfragen

Wem gehört KI-generierter Code, und darfst du ihn verwenden? Die Rechtslage ist je nach Land im Fluss, und dieses Handbuch ersetzt keine Rechtsberatung. Diese Punkte solltest du kennen:

- **Urheberrecht an KI-Ausgaben:** In vielen Rechtsordnungen (auch in Deutschland) setzt ein urheberrechtlich geschütztes Werk eine persönliche geistige Schöpfung eines Menschen voraus. Rein maschinell erzeugter Code ist deshalb unter Umständen nicht geschützt. Dein eigener kreativer Beitrag (Planung, Auswahl, Anpassung) kann eine Rolle spielen.
- **Nutzungsbedingungen des Anbieters:** Jeder Dienst regelt, was du mit den Ausgaben tun darfst. Lies sie bei kommerziellem Einsatz.
- **Übernommener Code:** Modelle können Codeschnipsel erzeugen, die bekanntem Open-Source-Code ähneln. Bei größeren Abschnitten kann eine Lizenz greifen. Manche Dienste bieten Filter oder Quellenhinweise.
- **Open-Source-Lizenzen deiner Abhängigkeiten:** MIT und BSD sind sehr freizügig, die GPL verlangt unter Umständen Offenlegung abgeleiteter Werke. Prüfe die Lizenz jedes eingesetzten Pakets (`pip show`).
- **Eigene Lizenz wählen:** Wenn du Code veröffentlichst, lege eine `LICENSE`-Datei bei. Ohne Lizenz darf ihn rechtlich niemand weiterverwenden.

Gut zu wissen

Praxis-Empfehlung: Für private Werkzeuge und Lernprojekte sind Lizenzfragen kaum relevant. Sobald du Code **verkaufst, an Kunden gibst oder in einer Firma einsetzt**, kläre die Lage mit dem Arbeitgeber oder einer Rechtsberatung.

6.8 Datenschutz

Alles, was du in einen KI-Dienst eingibst, wird an einen externen Anbieter übertragen. Deshalb gilt:

Gib nie ein, was nicht nach außen gehören darf:

- Passwörter, Keys, Tokens, private Schlüssel (`~/.ssh/id_*`)
- Personenbezogene Daten Dritter (Kundenlisten, Namen mit Adressen, Gesundheitsdaten)
- Betriebsgeheimnisse und vertraulichen Firmencode, sofern dein Arbeitgeber das nicht ausdrücklich erlaubt
- Interne Netzwerkdaten (IP-Pläne, Hostnamen, Logdateien mit Nutzerdaten)

Sicher arbeiten mit echten Daten

- **Anonymisiere** Beispieldaten: Ersetze Namen, Adressen und Nummern durch erfundene Werte.
- **Schicke Strukturen statt Inhalte:** „Eine CSV mit den Spalten Name, Datum, Betrag“ genügt oft, um Code zu schreiben.

- **Erzeuge Testdaten**, am besten mit dem Programm selbst.
- **Prüfe Einstellungen:** Viele Anbieter erlauben, die Verwendung von Gesprächen zum Training einzuschränken. Schaue in die Datenschutzeinstellungen.
- **Lokale Modelle:** Für sehr sensible Daten gibt es Modelle, die auf deinem eigenen Rechner laufen (zum Beispiel über Ollama oder llama.cpp). Sie sind meist weniger leistungsfähig, aber deine Daten verlassen den Rechner nicht.

Beispiel 6.10: Testdaten für eine Beispiel-CSV selbst erzeugen (keine echten Daten nötig)

```
import csv

with open("beispiel.csv", "w", newline="", encoding="utf-8") as f:
    schreiber = csv.writer(f)
    schreiber.writerow(["name", "datum", "betrag"])
    schreiber.writerow(["Max Muster", "2026-01-15", "42,50"])
    schreiber.writerow(["Erika Beispiel", "2026-02-03", "17,90"])
```

WARNUNG – Sicherheitsrisiko

Ein schneller `sed`- oder `awk`-Einzeiler auf einer **echten** Kundendatei ist **keine verlässliche Anonymisierung**: CSV-Felder können in Anführungszeichen stehen, Kommas oder Zeilenumbrüche enthalten, und ein Namensmuster erwischt nicht jede Schreibweise. Vor allem bleiben andere identifizierende Spalten (E-Mail, Adresse, Freitextfelder) komplett unangetastet. Für eine KI-Anfrage brauchst du in aller Regel ohnehin keine echten Daten, sondern nur deren **Struktur** – erzeuge sie stattdessen selbst, wie oben. Musst du wirklich mit einer echten Datei arbeiten, nutze eine CSV-Bibliothek mit einer vorab festgelegten Liste erlaubter Spalten (Allowlist) statt eines Text-Einzeilers, und prüfe Freitextfelder gesondert von Hand.

WARNUNG – Sicherheitsrisiko

Für personenbezogene Daten in Deutschland und der EU gelten die Datenschutz-Grundverordnung (DSGVO) und das Bundesdatenschutzgesetz. Wer solche Daten **beruflich** an einen KI-Dienst schickt, braucht dafür in der Regel eine Rechtsgrundlage und meist einen Auftragsverarbeitungsvertrag. Frage vorher die zuständige Stelle in deiner Organisation. Der sicherste Weg bleibt: echte Datensätze erst gar nicht in externe Chats kopieren.

6.9 Wann du KI-Code NICHT vertrauen darfst

KI-Code ist ein Entwurf, nie ein Urteil. In diesen Fällen brauchst du besondere Vorsicht bis hin zu **fachkundiger menschlicher Prüfung**:

Bereich	Warum kritisch
Kryptografie und Verschlüsselung	Kleine Fehler machen alles wertlos; nutze nur bewährte Bibliotheken
Anmeldung, Sitzungen, Rechtevergabe	Fehler öffnen Türen für Angreifer
Zahlungen und Finanzdaten	Hoher Schaden, rechtliche Pflichten
Medizin, Sicherheit, Verkehr, Industrie	Gefahr für Menschen
Betrieb im Internet	Ständige Angriffe auf offene Dienste
Systemadministration mit Root-Rechten	Ein falscher Befehl beschädigt das System
Datenlöschung, Migration, Backups	Fehler sind oft nicht umkehrbar

Bereich	Warum kritisch
Rechtliche, steuerliche Berechnungen	Falsche Annahmen mit realen Folgen

Faustregel: **Je höher Schaden und Unumkehrbarkeit, desto weniger Vertrauen und desto mehr menschliche Prüfung.** Für ein Spielprojekt reicht „läuft“. Für alles, was Menschen, Geld oder fremde Daten betrifft, reicht es nicht.

Übungen

Übung

1. Durchsuche eines deiner Projekte mit Beispiel 6.2 nach Geheimnissen. Gibt es Treffer? Wie würdest du sie beheben?
2. Lass dir von der KI drei gefährliche Terminalbefehle nennen und jeweils erklären, was sie anrichten – führe sie **nicht** aus.
3. Speichere Beispiel 6.5 als Datei in `/tmp` und führe es dort aus. Gib bei beiden Varianten (unsicher und sicher) die Testeingabe `x; echo GEHACKT` ein und vergleiche die Ausgabe mit Beispiel 6.5b.
4. Erzeuge mit `pruefe.sh` (Beispiel 6.8) eine Prüfroutine für ein eigenes Projekt und binde sie in deinen Ablauf vor jedem Commit ein.
5. Entscheide für fünf Projektideen, ob du KI-Code ungeprüft, geprüft oder nur mit fachkundiger Prüfung einsetzen würdest.

Lösungshinweis

Zu 3: Bei der unsicheren Variante erscheint `GEHACKT` als Ausgabe eines **eigenständig ausgeführten zweiten Befehls** – die Shell hat den Strichpunkt als Trenner gelesen. Bei der sicheren Variante bleibt `GEHACKT` Teil der (nicht existierenden) Dateinamens-Fehlermeldung; es wurde kein zweiter Befehl ausgeführt. Vergleiche genau diese beiden Fehlermeldungen aus Beispiel 6.5b.

Zusammenfassung

Zusammenfassung

- Keys gehören in Umgebungsvariablen, nie in Code, Chat oder Repository; bei Leck sofort widerrufen.
- Prüfe jeden vorgeschlagenen Befehl mit den drei Fragen; starte Agenten nur in Projektordnern.
- Typische Schwachstellen: Befehls-, SQL- und Pfad-Einschleusung sowie XSS; offene Server und selbstgebaute Anmeldung.
- Eine feste Prüfroutine (Tests, Linter, Geheimnis-Suche, `git diff`) gehört vor jeden Commit.
- Prüfe Pakete vor der Installation; bevorzuge die Standardbibliothek.
- Lizenz und Datenschutz: Sobald es beruflich oder kommerziell wird, kläre Rechtsfragen fachkundig.
- Bei Krypto, Anmeldung, Zahlungen und sicherheitskritischen Systemen gilt: nie ungeprüft vertrauen.

Kapitel 7: Typische Fallstricke und Troubleshooting

Lernziele

- Du erkennst Halluzinationen und überprüfst Behauptungen der KI.
- Du gehst mit veraltetem Code und Versionskonflikten um.
- Du löst Abhängigkeitsprobleme systematisch.
- Du gehst konstruktiv mit „Es funktioniert, aber ich verstehe es nicht“ um.

7.1 Halluzinationen

Eine **Halluzination** ist eine überzeugend klingende, aber falsche Aussage der KI. Beim Programmieren treten sie meist in diesen Formen auf:

Form	Beispiel
Erfundene Funktion	`datei.lese_alles_sicher()` – gibt es nicht
Erfundene Option	`grep --nur-treffer` – existiert nicht
Erfundenes Paket	`pip install superparser-pro` – unbekannt oder gefährlich
Falsche Version	Funktion existiert erst in einer neueren Version
Erfundene Quelle	Verweis auf eine Dokumentationsseite, die es nie gab
Falsche Selbstauskunft	„Ich habe den Code getestet“ – ohne Ausführung

So überprüfst du Behauptungen

1. **Existiert der Befehl?** Nutze die eingebaute Hilfe.
2. **Existiert die Funktion?** Frage die Python-Hilfe ab.
3. **Stimmt die Option?** Schau in die Handbuchseite.
4. **Läuft es wirklich?** Nur Ausführen beweist es.

Beispiel 7.1: Behauptungen schnell nachprüfen

```
$ command -v jq                                  # gibt es das Programm?
$ man grep | less                                # Optionen nachlesen (q beendet)
$ grep --help | head -30                         # Kurzhilfe
$ python3 -c "import pathlib; help(pathlib.Path.read_text)"
$ python3 -c "import json; print(dir(json))"     # welche Namen gibt es?
$ apt-cache policy jq                            # Paket und Version in Debian/Ubuntu
```

Gegenmaßnahmen im Prompt

- „Wenn du dir bei einer Funktion oder Option nicht sicher bist, sage es ausdrücklich.“
- „Nenne die Python-Version, ab der die Funktion existiert.“
- „Erfinde keine Pakete. Wenn die Standardbibliothek reicht, nutze sie.“
- „Verweise auf offizielle Dokumentation und nenne das Stichwort, nach dem ich suchen kann.“

> **Gut zu wissen**
>
> Frage die KI nie: „Bist du sicher?“ als einzigen Test. Manche Modelle rudern dann auch bei richtigen Antworten zurück. Besser: „Wie kann ich das selbst überprüfen?“ Das liefert dir einen Testbefehl.

7.2 Veralteter Code und Versionskonflikte

Modelle lernen aus Daten bis zu einem Stichtag. Bibliotheken ändern sich danach weiter. Die Folge: Die KI schlägt Aufrufe vor, die in der neuen Version geändert oder entfernt wurden, oder umgekehrt Funktionen einer neueren Version, die bei dir noch nicht existieren.

Typische Symptome

- `AttributeError: module 'x' has no attribute 'y'`
- `DeprecationWarning` oder `FutureWarning` in der Ausgabe
- Beispiele aus Tutorials, die nicht mehr laufen
- Befehle mit Optionen, die deine Programmversion nicht kennt

Vorgehen

Beispiel 7.2: Versionen feststellen

```
$ python3 --version
$ pip show flask | head -3          # Name, Version
$ pip list --outdated               # veraltete Pakete (in der venv)
$ node --version                    # falls JavaScript-Tooling im Spiel ist
```

Gib der KI die **tatsächlichen Versionen** und bitte um Code für genau diese:

Beispiel 7.3: Versionsbewusster Prompt

```
Ich nutze Python 3.11 und Flask 3.0.2 unter Debian 12. Dein Code verwendet
eine Funktion, die dort offenbar nicht existiert (Fehlermeldung unten).
Prüfe, ob sie in meiner Version existiert; falls nicht, nenne die korrekte
Alternative für genau diese Versionen und verweise auf den Dokumentationsabschnitt.
```

> **TIPP – Profi-Trick**
>
> Wenn die KI Webzugriff hat, bitte sie ausdrücklich, die aktuelle offizielle Dokumentation nachzuschlagen. Ohne Webzugriff: Kopiere den relevanten Abschnitt der Dokumentation selbst in den Chat.

Pin: Versionen festlegen

Damit ein Projekt auch in Monaten noch läuft, **friere** du die Versionen ein (siehe Beispiel 2.7). So sind Fehler später reproduzierbar, und Updates machst du bewusst, nicht aus Versehen.

7.3 Abhängigkeitskonflikte

Ein **Abhängigkeitskonflikt** entsteht, wenn zwei Pakete verschiedene Versionen eines dritten verlangen oder wenn Pakete mit deiner Python-Version nicht zusammenpassen.

Häufige Fehlermeldungen

Meldung	Bedeutung und Lösung
`externally-managed-environment`	System-Python gesperrt. Lösung: venv verwenden
`ModuleNotFoundError: No module named 'x'`	Paket fehlt oder falsche Umgebung aktiv. `which python` prüfen
`ERROR: Cannot install a and b because these package versions have conflicting dependencies`	Versionskonflikt. Eine Version festlegen oder Paket weglassen
`error: command 'gcc' failed` beim Installieren	Compiler oder Header fehlen: `sudo apt install build-essential python3-dev`
`No matching distribution found`	Paketname falsch oder Python zu alt/neu
`Permission denied` bei `pip install`	Nicht mit `sudo`; venv nutzen

Systematische Fehlersuche

Beispiel 7.4: Die richtige Umgebung prüfen

```
$ which python3 pip                 # Aus der venv? Pfad sollte auf .venv zeigen
$ python3 -c "import sys; print(sys.prefix)"
$ python3 -m pip list               # Was ist hier installiert?
$ python3 -m pip check              # Meldet kaputte Abhängigkeiten
```

Wenn gar nichts mehr hilft, ist der Neustart die schnellste Lösung:

Beispiel 7.5: Virtuelle Umgebung sauber neu aufbauen

```
$ deactivate 2>/dev/null
$ rm -rf .venv
$ python3 -m venv .venv
$ source .venv/bin/activate
(.venv) $ pip install -r requirements.txt
```

Das kostet eine Minute und löst erstaunlich viele „unerklärliche“ Probleme. Weil die venv nur Wegwerf-Material enthält (und in `.gitignore` steht), ist das gefahrlos.

> **TIPP – Profi-Trick**
>
> Aktiviere bei Problemen zuerst **den Verdacht auf die Umgebung**: Falsche venv aktiv? Falscher Python? Paket in einer anderen Umgebung installiert? Das ist bei Anfängern die häufigste Ursache.

7.4 „Es funktioniert, aber ich verstehe es nicht“

Das ist die typische Falle des Vibe Codings: Der Code läuft, und du hast keine Ahnung, warum. Solange nichts schiefgeht, ist das kein Problem. Sobald etwas kaputtgeht, schon.

Warum das gefährlich ist

- Du kannst Fehler nicht eingrenzen.
- Du merkst nicht, wenn die KI etwas Riskantes eingebaut hat.
- Jede Erweiterung wird zum Glücksspiel.

Die Verständnis-Leiter

Du musst nicht alles verstehen, aber du solltest **stufenweise** verstehen:

1. **Zweck:** Was tut das Programm als Ganzes? (Ein Satz.)
2. **Aufbau:** Welche Teile (Funktionen, Dateien) gibt es und wofür?
3. **Ablauf:** In welcher Reihenfolge geschieht was?
4. **Details:** Was tut diese Zeile? Warum so?

Beginne oben. Meist genügt Stufe 2 bis 3, um Fehler gezielt zu suchen.

Beispiel 7.6: Prompt für Verständnis in Stufen

```
Erkläre mir dieses Programm in vier Stufen, jeweils kurz:
1. Zweck in einem Satz
2. Aufbau: Liste der Funktionen mit je einem Satz
3. Ablauf: Was passiert vom Start bis zur Ausgabe, in nummerierten Schritten
4. Drei Stellen, an denen es am ehesten schiefgehen kann
Ich bin Anfänger und kenne nur Linux-Grundlagen.
```

Techniken, die Verständnis erzeugen

- **Kommentieren lassen:** „Füge verständliche Kommentare ein, die erklären, warum, nicht nur was."
- **Kleine Änderung selbst machen:** Ändere eine Zahl oder einen Text und beobachte die Folge.
- **Zeile auskommentieren:** Mit `#` in Python lässt du eine Zeile weg und siehst, was fehlt.
- **Spielzeug-Version:** Lass dir die Kernidee in fünf Zeilen als Mini-Beispiel zeigen.
- **Erklären lernen:** Erkläre das Programm der KI in eigenen Worten und lass dich korrigieren. Das ist eine der wirksamsten Lernmethoden überhaupt.
- **Debugger oder Diagnose-Ausgaben:** Schau zu, wie sich Werte beim Ablauf ändern (Beispiel 3.11).

> **Gut zu wissen**
>
> Ein Verständnis-Ziel für Anfänger: Du kannst für jedes deiner Programme in zwei Minuten jemandem erklären, **was es tut, aus welchen Teilen es besteht und wo du bei einem Fehler zuerst suchen würdest.** Das reicht für den Anfang weit.

7.5 Weitere Fallstricke

Die KI ändert zu viel

Symptom: Du bittest um eine Kleinigkeit und bekommst eine umgebaute Datei. Gegenmittel: „Ändere nur X, sonst nichts", „Zeige nur das Diff", Prüfung mit `git diff`, kleinere Schritte.

Die Endlosschleife

Symptom: Fix A bricht B, Fix B bricht A. Gegenmittel: Zurück zum letzten guten Stand (`git restore .`), Ursache **erklären** lassen, Problem isolieren (Abschnitt 3.7).

Übermäßige Komplexität

Die KI baut gern große Strukturen: Klassen, Frameworks, Konfigurationsdateien für ein 30-Zeilen-Problem. Bitte um „die einfachste Lösung, die funktioniert" und „eine Datei, nur Standardbibliothek".

Zu viel Vertrauen in grüne Tests

Die KI schreibt manchmal Tests, die ihr eigenes Verhalten bestätigen, statt das Soll zu prüfen. Prüfe die erwarteten Werte selbst nach: Stimmt „erwartet“ mit dem überein, was **du** möchtest?

Stilles Scheitern

Code, der Fehler verschluckt (`except: pass`), meldet nie ein Problem. Suche solche Stellen gezielt:

Beispiel 7.7: Verdächtige Fehlerbehandlung aufspüren

```
$ rg -n "except\s*:|except Exception:\s*$|pass$" --glob '*.py' .
$ rg -n "2>/dev/null|\|\| true" --glob '*.sh' .
```

Nicht jeder Treffer ist falsch, aber jeder ist eine Frage wert: „Warum wird dieser Fehler ignoriert?“

Zeilenenden und Zeichensatz

Beim Kopieren aus Webseiten oder Dokumenten schleichen sich unsichtbare Zeichen und Windows-Zeilenenden ein. Typische Fehler: `bad interpreter: /bin/bash^M` oder seltsame Anführungszeichen.

Beispiel 7.8: Zeilenenden und Zeichen prüfen und reparieren

```
$ file skript.sh                      # zeigt "CRLF line terminators", falls betroffen
$ sed -i 's/\r$//' skript.sh          # entfernt Windows-Zeilenenden
$ grep -nP '[^\x00-\x7F]' skript.sh   # findet Nicht-ASCII-Zeichen (Umlaute, ’ usw.)
```

Einrückung in Python

Mischung aus Tabs und Leerzeichen erzeugt `IndentationError`. Nutze im Editor durchgehend vier Leerzeichen; lass die KI den Code neu formatieren oder nutze `ruff format`.

7.6 Ein Fahrplan für jedes Problem

1. **Ruhig bleiben, Meldung lesen** – vollständig.
2. **Reproduzieren:** Tritt es jedes Mal auf? Mit welchem Befehl?
3. **Eingrenzen:** Was hat sich seit dem letzten funktionierenden Stand geändert? (`git diff`, `git stash`)
4. **Umgebung prüfen:** Versionen, venv, Rechte, Pfade.
5. **Erklären lassen,** dann erst Fix vorschlagen lassen.
6. **Ein Fix auf einmal,** sofort testen.
7. **Dokumentieren:** Ursache und Lösung ins Lerntagebuch.

> **TIPP – Profi-Trick**
>
> `git bisect` findet automatisch den Commit, der einen Fehler eingeführt hat. Dafür brauchst du viele kleine Commits – ein weiterer Grund für die Regel „Ein Schritt = ein Commit“. Frage die KI nach einer Anleitung, sobald du es brauchst.

Übungen

> **Übung**

1. Bitte die KI um eine Funktion, die es nicht gibt (erfinde einen Namen). Wie reagiert sie? Prüfe mit `help()` oder `dir()`.
2. Erzeuge absichtlich einen `ModuleNotFoundError` (Import eines nicht installierten Pakets) und löse ihn mit venv und `pip install`.
3. Baue die venv eines Projekts nach Beispiel 7.5 neu auf.
4. Nimm ein Programm, das du nicht verstehst, und arbeite die Verständnis-Leiter (Beispiel 7.6) durch. Erkläre es dann der KI in eigenen Worten.
5. Suche mit Beispiel 7.7 in einem Projekt nach stillem Scheitern und bewerte jeden Treffer.

Lösungshinweis

Zu 2: `python3 -c "import notexistingpkg"` erzeugt `ModuleNotFoundError`. Mit aktivierter venv installierst du ein echtes Paket mit `pip install paketname`. Wichtig: Prüfe vorher mit `which python3`, dass die venv aktiv ist.

Zusammenfassung

Zusammenfassung

- Halluzinationen erkennst du nur durch Gegenprüfung: `command -v`, `man`, `help()`, Ausführen.
- Gib der KI die tatsächlichen Versionen und friere Abhängigkeiten ein.
- Bei Abhängigkeitsproblemen zuerst die Umgebung prüfen; eine frische venv löst vieles.
- Verständnis wächst stufenweise: Zweck, Aufbau, Ablauf, Details.
- Typische Probleme: zu große Änderungen, Endlosschleifen, Überkomplexität, stilles Scheitern, Zeilenenden.
- Der Fahrplan: lesen, reproduzieren, eingrenzen, Umgebung prüfen, erklären lassen, ein Fix, dokumentieren.

Kapitel 8: Nächste Schritte

> **Lernziele**
>
> - Du planst deinen persönlichen Lernpfad vom Vibe Coder zum sicheren Entwickler.
> - Du kennst gute Quellen und Gemeinschaften zum Weiterlernen.
> - Du wählst Folgeprojekte passend zu deinem Interesse.
> - Du hast eine Checkliste für deinen Alltag als Vibe Coder.

8.1 Wo stehst du jetzt?

Wenn du dieses Handbuch durchgearbeitet hast, kannst du:

- eine Umgebung mit Editor, Python, Git und KI-Assistent einrichten,
- Aufgaben in kleine Schritte zerlegen und mit der KI umsetzen,
- Fehlermeldungen auswerten und Code reviewen lassen,
- drei Arten von Projekten bauen (CLI, Web, Automatisierung),
- Risiken erkennen: Keys, Befehle, Pakete, Daten.

Das ist ein solides Fundament. Der Unterschied zu einem erfahrenen Entwickler liegt vor allem im **Urteilsvermögen**: Welche Lösung ist einfach, robust und wartbar? Dieses Gefühl entsteht durch Praxis und durch immer tieferes Verstehen.

8.2 Der Lernpfad in vier Stufen

Stufe	Ziel	Typische Aktivitäten	Zeitrahmen (Richtwert)
1. Vibe Coder	Funktionierende Werkzeuge bauen	Dieses Handbuch, kleine Projekte	2 bis 4 Wochen
2. Verstehender Nutzer	Code lesen und anpassen	Eigene Änderungen, Tests, Debugging	2 bis 3 Monate
3. Selbstständiger Entwickler	Programme selbst entwerfen	Python-Grundlagen systematisch, Datenstrukturen, Module	6 bis 12 Monate
4. Fortgeschritten	Größere Systeme, Qualität	Architektur, Datenbanken, Sicherheit, Teamarbeit	fortlaufend

Die Zeitangaben sind grobe Richtwerte und hängen stark von deiner Übungszeit ab. Wichtiger als die Dauer ist die Regelmäßigkeit: lieber 30 Minuten täglich als ein Marathon pro Monat.

Was du auf Stufe 2 als Nächstes lernen solltest

Diese Grundlagen machen dich unabhängiger von der KI und deine Anfragen präziser:

1. **Variablen, Datentypen** (Text, Zahlen, Listen, Wörterbücher)
2. **Kontrollfluss:** `if`, `for`, `while`
3. **Funktionen und Module**
4. **Dateien lesen und schreiben**
5. **Fehlerbehandlung** (`try`/`except`)

6. **Tests schreiben** und ausführen
7. **Git im Alltag:** Branches, Merge, Remote-Repositories
8. **Debuggen:** Diagnose-Ausgaben, Debugger

Ein einfacher Lernrhythmus

Beispiel 8.1: Wochenplan für Stufe 2

```
- Montag, 30 min: Ein Konzept lernen (z. B. Schleifen) – KI erklärt mit Alltagsbeispiel
- Mittwoch, 45 min: Eine Mini-Aufgabe OHNE KI-Code lösen, dann mit KI vergleichen
- Freitag, 45 min: Am eigenen Projekt eine kleine Erweiterung bauen
- Sonntag, 10 min: Lerntagebuch schreiben und Wochenrückblick
```

> **TIPP – Profi-Trick**
>
> **Die 70/30-Regel zum Lernen:** Versuche eine Aufgabe zuerst selbst, mindestens ein paar Minuten lang, bevor du die KI fragst. Dann lernst du aus der Differenz zwischen deinem Versuch und ihrer Lösung. Wer sofort fragt, lernt deutlich weniger. Lass dir außerdem Hinweise statt Komplettlösungen geben: „Gib mir nur einen Tipp, nicht die Lösung."

8.3 Die KI als Lehrer einsetzen

Du kannst die KI gezielt als geduldigen Nachhilfelehrer verwenden:

Beispiel 8.2: Der KI-Tutor-Prompt

```
Du bist mein Python-Tutor. Ich bin Linux-Anwender mit Terminal-Erfahrung,
aber Programmieranfänger. Gib mir ein Mini-Programmierproblem zum Thema
"Schleifen". Löse es NICHT, sondern warte auf meinen Lösungsversuch.
Gib mir danach Rückmeldung: was gut ist, was fehlt, und einen Hinweis statt
der fertigen Lösung. Steigere die Schwierigkeit langsam.
```

Weitere Lernformate:

- **Code-Lesen:** „Zeige mir ein 20-Zeilen-Programm und frage mich, was es tut."
- **Fehlersuche:** „Baue einen versteckten Fehler ein, und ich suche ihn."
- **Erklären-lassen:** „Prüfe mein Verständnis mit fünf Fragen zu diesem Kapitel."
- **Projektbegleitung:** „Begleite mich beim Bau von X, aber schreibe nur den Code, den ich ausdrücklich verlange."

8.4 Folgeprojekte nach Interesse

Wähle Projekte, die du **wirklich nutzen** würdest. Das motiviert am meisten.

Interesse	Projektidee	Neu zu lernen
Systemadministration	Backup-Skript mit Rotation und Bericht per Mail	Bash, rsync, Fehlerbehandlung, cron/systemd
Heimnetz	Dashboard mit Geräten, Speicher, Temperatur	Systeminfos auslesen, kleine Weboberfläche
Daten	CSV-Auswertung mit Diagrammen	Python-Bibliotheken für Daten und Grafik

Interesse	Projektidee	Neu zu lernen
Web	Persönliche Webseite oder Blog (statisch)	HTML, CSS, Hosting, Domain
Alltag	Haushaltsbuch im Terminal	Dateiformate, Datenbank (SQLite), Tests
Medien	Bildersortierer anhand der Aufnahmedaten	Metadaten, Dateioperationen
KI	Eigenes Skript, das einen KI-Dienst über die API anspricht	HTTP, JSON, Umgang mit Keys (Kapitel 6)

Ein Projekt mit KI-API

Der nächste logische Schritt: Dein Programm ruft selbst eine KI auf. Dafür liest du den Key aus der Umgebung (Beispiel 6.1) und sendest eine Anfrage über HTTP. Die genaue Anleitung steht in der Dokumentation deines Anbieters; frage dort nach dem aktuellen Beispiel, weil sich Modellnamen und Schnittstellen ändern. Beachte Kosten (Ausgabenlimit!), Datenschutz (Kapitel 6.8) und Fehlerbehandlung bei Netzwerkproblemen.

> **WARNUNG – Sicherheitsrisiko**
>
> Programme, die automatisch KI-Dienste aufrufen, können durch Fehler (Endlosschleifen) hohe Kosten verursachen. Setze ein Ausgabenlimit im Konto, begrenze Wiederholungen im Code und teste zuerst mit wenigen kleinen Anfragen.

8.5 Ressourcen zum Weiterlernen

Die Qualität und Verfügbarkeit von Angeboten ändert sich; prüfe Aktualität und Qualität selbst. Bewährte Anlaufstellen:

Offizielle Dokumentation (zuverlässigste Quelle)

- **Python:** docs.python.org, vor allem das offizielle Tutorial (auch auf Deutsch verfügbar)
- **Git:** git-scm.com, das Buch „Pro Git“ ist dort kostenlos und auf Deutsch lesbar
- **MDN Web Docs:** developer.mozilla.org, die beste Referenz für HTML, CSS, JavaScript
- **Bash:** `man bash` und das GNU Bash Reference Manual
- **Flask:** flask.palletsprojects.com
- **Dokumentation deines KI-Werkzeugs:** Funktionen und Befehle ändern sich schnell; lies sie regelmäßig

Lernplattformen und Kurse

- Das kostenlose Programmierangebot **freeCodeCamp**
- Das Einsteigerprogramm **CS50** der Harvard University (online frei zugänglich)
- Interaktive Python-Kurse und Übungsplattformen (zum Beispiel Exercism)
- **VHS, IHK und Weiterbildungsträger** bieten Kurse für Einsteiger in Präsenz und online

Bücher

- „Automate the Boring Stuff with Python“ von Al Sweigart: genau für Menschen, die Alltagsaufgaben automatisieren möchten (online frei lesbar, auch auf Deutsch erschienen)
- „Pro Git“ von Scott Chacon und Ben Straub
- „The Linux Command Line“ von William Shotts (falls du Terminalkenntnisse vertiefen willst)

Gemeinschaften

- **Foren und Q&A-Seiten** (zum Beispiel Stack Overflow, deutsche Linux-Foren wie ubuntuusers.de)
- **Subreddits und Discord-Server** zu Python, Linux und KI-gestütztem Programmieren
- **Lokale Gruppen:** Linux-User-Groups, Chaos Computer Club Treffs, Meetups, Repair-Cafés mit Programmier-Tisch
- **Open-Source-Projekte:** Beiträge zu Dokumentation und kleinen Fehlern sind ein hervorragender Einstieg

> **TIPP – Profi-Trick**
>
> Wenn du in einem Forum fragst, nutze das Muster aus Beispiel 3.9: Ziel, Befehl, vollständige Fehlermeldung, Umgebung, und schreibe, was du schon versucht hast. Damit bekommst du deutlich bessere Antworten, und du zeigst, dass du dich bemüht hast.

8.6 Berufliche Perspektiven realistisch einschätzen

Vibe Coding ist ein starkes Werkzeug für private Projekte, Prototypen und Automatisierung am Arbeitsplatz. Als alleinige Grundlage für Software, die Kunden bezahlen, ist es dünn. Wer beruflich einsteigen möchte, braucht zusätzlich:

- **Grundlagenwissen** in einer Programmiersprache, Datenstrukturen und Entwurfsprinzipien,
- **Fähigkeit zur Fehlersuche ohne KI,**
- **Sicherheitsbewusstsein** (Kapitel 6),
- **Teamwerkzeuge** (Git-Workflows, Code-Reviews, Tickets),
- **Nachweise** in Form von Projekten, die du zeigen und erklären kannst.

In der IT-Systemintegration, in der Administration und im Support sind Skripte und Automatisierung schon heute ein großer Vorteil. Auch hier gilt: Wer die Werkzeuge versteht und Verantwortung für Qualität und Sicherheit übernimmt, ist wertvoller als jemand, der nur Code erzeugen lässt.

> **Gut zu wissen**
>
> Ein Portfolio muss nicht groß sein: drei saubere Projekte mit README, Tests und klarer Beschreibung, was du selbst gemacht hast und was die KI beigetragen hat, wirken überzeugender als zehn halbfertige. Ehrlichkeit über KI-Unterstützung wird in der Branche zunehmend erwartet.

8.7 Deine Alltags-Checkliste

Drucke sie aus oder lege sie in `~/vibe/notizen/checkliste.md` ab:

Beispiel 8.3: Checkliste für jedes Projekt

```
## Vor dem Start
- [ ] Ziel in einem Satz, MVP und Plan mit Testbefehlen
- [ ] Projektordner, Git-Repository, venv, .gitignore
- [ ] PROJEKT.md mit Rahmenbedingungen

## Während der Arbeit
- [ ] Kleiner Schritt, ein Test, ein Commit
- [ ] git diff gelesen
- [ ] Keine Geheimnisse im Chat oder Code
- [ ] Riskante Befehle erklärt und geprüft

## Vor dem Einsatz
- [ ] Normal-, Fehler- und Randfälle getestet
- [ ] Trockenlauf bei Programmen, die Daten verändern
- [ ] Backup vorhanden
- [ ] Abhängigkeiten geprüft, requirements.txt aktuell
- [ ] README geschrieben

## Danach
- [ ] Lerntagebuch ergänzt
- [ ] Offene Fragen notiert
```

Schlusswort

Vibe Coding senkt die Einstiegshürde, nimmt dir aber nicht die Verantwortung ab. Du hast jetzt die Werkzeuge: einen Arbeitsablauf, der kleine Schritte belohnt, ein Sicherheitsnetz mit Git, Prüfroutinen gegen die häufigsten Fehler und die Haltung, KI-Code als Entwurf zu behandeln. Baue Dinge, die du wirklich brauchst. Mach Fehler in sicherer Umgebung. Frage dich bei jedem Programm: „Verstehe ich, was es tut?“, und lerne mit jedem Projekt ein Stück mehr.

Viel Freude und Erfolg beim Bauen.

Übungen

Übung

1. Lege deinen persönlichen Lernplan für die nächsten vier Wochen an (Beispiel 8.1).
2. Wähle ein Folgeprojekt aus Tabelle 8.4 und schreibe Plan und `PROJEKT.md`.
3. Starte eine Tutor-Sitzung nach Beispiel 8.2 zu einem Thema, das du noch nicht verstehst.
4. Erstelle die Checkliste aus Beispiel 8.3 und wende sie auf eines deiner bisherigen Projekte an.
5. Schreibe eine README für dein bestes Projekt, die auch jemand ohne Vorwissen versteht.

Zusammenfassung

Zusammenfassung

- Dein Lernpfad führt vom Vibe Coder über den verstehenden Nutzer zum selbstständigen Entwickler.
- Versuche Aufgaben zuerst selbst; nutze die KI als Tutor, nicht nur als Lieferant.
- Wähle Folgeprojekte, die du wirklich nutzt; steigere Schwierigkeit und Verantwortung langsam.
- Offizielle Dokumentation, freie Kurse, Bücher und Gemeinschaften helfen beim Weiterlernen.
- Beruflich brauchst du zusätzlich Grundlagenwissen, Sicherheitsbewusstsein und herzeigbare Projekte.

- Eine feste Checkliste macht gute Praxis zur Gewohnheit.

Glossar

Abhängigkeit – Fremdes Programmpaket, das dein Projekt zum Laufen braucht.

Agent (KI-Agent) – KI-Werkzeug, das selbstständig Schritte ausführt, etwa Dateien lesen, ändern und Befehle ausführen.

API – Schnittstelle, über die Programme miteinander oder mit Diensten sprechen.

API-Key – Geheimer Zugangsschlüssel zu einem Dienst; wer ihn kennt, nutzt den Dienst auf deine Rechnung.

Argument – Wert, den du einem Programm oder einer Funktion beim Aufruf mitgibst.

Backend – Teil einer Anwendung, der auf dem Server läuft (Daten, Logik).

Bash – Weit verbreitete Linux-Shell und Skriptsprache.

Branch – Eigener Entwicklungszweig in Git.

Code-Review – Kritische Durchsicht von Code auf Fehler, Sicherheit und Lesbarkeit.

Commit – Gespeicherter Schnappschuss des Projekts in Git.

cron – Klassischer Dienst zum zeitgesteuerten Ausführen von Befehlen.

CSS – Sprache für das Aussehen von Webseiten.

Debugging – Fehler finden und beheben.

Diff – Gegenüberstellung von Änderungen (neu mit `+`, entfernt mit `-`).

DSGVO – Datenschutz-Grundverordnung der EU.

Exit-Code – Zahl, die ein Programm beim Beenden zurückgibt; 0 bedeutet Erfolg.

Flask – Kleines Python-Framework für Webanwendungen.

Framework – Fertiges Gerüst, das viel Grundarbeit abnimmt und Regeln vorgibt.

Funktion – Benannter, wiederverwendbarer Programmbaustein.

Git – Werkzeug zur Versionskontrolle.

Halluzination – Überzeugend klingende, aber falsche Ausgabe einer KI.

Hash – Digitaler Fingerabdruck eines Dateiinhalts (zum Beispiel SHA-256).

HTML – Auszeichnungssprache für den Inhalt von Webseiten.

IDE – Entwicklungsumgebung: Editor mit Zusatzfunktionen für Programmierer.

Iteration – Wiederholter Durchlauf aus Ändern, Testen und Verbessern.

JavaScript – Programmiersprache für Verhalten in Webseiten.

JSON – Textformat für strukturierte Daten.

Kontextfenster – Textmenge, die ein KI-Modell pro Gespräch gleichzeitig berücksichtigen kann.

Linter – Werkzeug, das Code automatisch auf typische Fehler und Stilprobleme prüft.

LLM – Large Language Model, großes Sprachmodell.

localStorage – Speicher im Browser für kleine Datenmengen.

MVP – Minimal funktionsfähige Version eines Projekts.

Neovim – Terminal-Editor mit Modi und hoher Anpassbarkeit.

Path Traversal – Angriff, bei dem Eingaben wie `../` aus einem erlaubten Ordner ausbrechen.

Prompt – Anweisung oder Frage an eine KI.

Repository – Projektordner samt Git-Verlauf.

Schleife – Programmstruktur, die etwas mehrfach wiederholt.

Shell – Programm, das Terminalbefehle entgegennimmt (zum Beispiel Bash).

SQL – Sprache für Datenbankabfragen.

SQL-Injection – Angriff, bei dem Eingaben Datenbankbefehle einschleusen.

Standardbibliothek – Sammlung von Modulen, die mit der Programmiersprache mitgeliefert wird.

systemd-Timer – Zeitsteuerung von Diensten unter modernen Linux-Systemen.

Test – Kleines Programm, das prüft, ob Code das Richtige tut.

Token – Text-Baustein, in dem ein Sprachmodell rechnet und abrechnet.

Traceback – Python-Fehlerbericht mit dem Weg zur Fehlerstelle.

Trockenlauf – Probelauf, der nur anzeigt, was passieren würde.

Umgebungsvariable – Benannter Wert, den die Umgebung an Programme weitergibt.

Versionskontrolle – Verfahren, um Änderungen nachvollziehbar zu speichern und zurückzunehmen.

venv – Virtuelle Python-Umgebung mit eigenen Paketen pro Projekt.

Vibe Coding – Software durch Beschreibung in natürlicher Sprache mit einer KI entwickeln.

XSS – Cross-Site-Scripting: Einschleusen von Schadcode über Nutzertexte in Webseiten.

Verzeichnis der Codebeispiele

Beispiel 1.1: Eine erste Anfrage an die KI … Seite 11
Beispiel 1.2: Typische Antwort – ein Hallo-Programm … Seite 11
Beispiel 1.3: Programm starten … Seite 11
Beispiel 2.1: Vorhandene Werkzeuge prüfen … Seite 13
Beispiel 2.2: Grundpakete unter Debian/Ubuntu … Seite 13
Beispiel 2.3: Grundpakete unter Fedora … Seite 14
Beispiel 2.4: Grundpakete unter Arch … Seite 14
Beispiel 2.2b: fd/fdfind robust verknüpfen … Seite 14
Beispiel 2.5: Arbeitsordner anlegen … Seite 14
Beispiel 2.6: Virtuelle Umgebung anlegen und aktivieren … Seite 15
Beispiel 2.7: Abhängigkeiten festhalten und wiederherstellen … Seite 15
Beispiel 2.8: VS Code installieren (Varianten) … Seite 16
Beispiel 2.9: Ordner in VS Code öffnen … Seite 16
Beispiel 2.10: Neovim installieren und das Tutorial starten … Seite 16
Beispiel 2.11: Typischer Ablauf mit einem Terminal-Agenten … Seite 17
Beispiel 2.12: API-Key sicher ablegen … Seite 18
Beispiel 2.13: Key in die Sitzung laden und prüfen … Seite 18
Beispiel 2.14: Git konfigurieren … Seite 18
Beispiel 2.15: Erstes Repository mit erstem Commit … Seite 19
Beispiel 2.16: Eine sinnvolle .gitignore für Python-Projekte … Seite 19
Beispiel 2.17: Skript "neues-projekt" zum Anlegen neuer Projekte … Seite 20
Beispiel 2.18: Skript installieren und ausprobieren … Seite 20
Beispiel 3.1: Planungs-Prompt – erst planen, dann bauen … Seite 24
Beispiel 3.2: Projektnotiz PROJEKT.md als Gedächtnis … Seite 24
Beispiel 3.3: Prompt für einen kleinen, klaren Schritt … Seite 24
Beispiel 3.4: dupfinder.py, Schritt 1 – Dateien auflisten … Seite 25
Beispiel 3.5: Schritt 1 testen … Seite 26
Beispiel 3.6: Funktionierenden Stand sichern … Seite 26
Beispiel 3.7: Erweiterung mit klarer Eingrenzung … Seite 26
Beispiel 3.8: Änderungen der KI mit git diff kontrollieren … Seite 26
Beispiel 3.9: Vorlage für eine Fehleranfrage … Seite 27
Beispiel 3.10: Fehlerausgabe in eine Datei umleiten … Seite 27
Beispiel 3.11: Diagnose-Ausgaben mit logging … Seite 28
Beispiel 3.12: Projektstruktur kompakt für die KI ausgeben … Seite 28
Beispiel 3.13: Gezielte Auszüge statt ganzer Dateien … Seite 28
Beispiel 3.14: Lerntagebuch-Eintrag … Seite 29
Beispiel 4.1: Ein zu vager Prompt … Seite 31
Beispiel 4.2: Ein präziser Prompt … Seite 31
Beispiel 4.3: Rückfragen erzwingen … Seite 32
Beispiel 4.4: Mit Beispiel-Ein- und Ausgabe arbeiten … Seite 32
Beispiel 4.5: Selbstprüfung anfordern … Seite 32
Beispiel 4.6: Umgebungsinfos für die KI sammeln … Seite 33
Beispiel 4.7: Mehrere Dateien sauber übergeben … Seite 33
Beispiel 4.8: Traceback richtig lesen … Seite 33
Beispiel 4.9: Typische Shell-Fehler und ihre Bedeutung … Seite 33
Beispiel 4.10: Relevante Logzeilen herausfiltern … Seite 34
Beispiel 4.11: Review-Prompt … Seite 34
Beispiel 4.12: Statische Prüfwerkzeuge laufen lassen … Seite 35
Beispiel 4.13: Ein einfacher Test mit unittest (Standardbibliothek) … Seite 35
Beispiel 4.14: Tests ausführen … Seite 35
Beispiel 4.15: Tests vor der Änderung anfordern … Seite 35
Beispiel 5.1: Projekt anlegen … Seite 37

Beispiel 5.2: Prompt für das Gruppieren … Seite 38
Beispiel 5.3: dupfinder.py – vollständige Fassung … Seite 39
Beispiel 5.4: Testordner mit bekannten Duplikaten erzeugen … Seite 40
Beispiel 5.5: tests/test_dupfinder.py … Seite 41
Beispiel 5.6: Tests ausführen und committen … Seite 41
Beispiel 5.7: dupfinder wie ein echtes Linux-Programm installieren … Seite 42
Beispiel 5.8: Prompt für die Aufgabenliste … Seite 43
Beispiel 5.9: index.html – Aufgabenliste (eine Datei) … Seite 44
Beispiel 5.10: Seite lokal ansehen … Seite 45
Beispiel 5.11: Zum Vergleich – der ungefilterte Aufruf (bewusst, mit Erklärung) … Seite 45
Beispiel 5.12: Flask installieren … Seite 45
Beispiel 5.13: app.py – minimaler Flask-Server mit JSON-Schnittstelle … Seite 46
Beispiel 5.14: Server starten und mit curl testen … Seite 46
Beispiel 5.15: Prompt für das Aufräumskript … Seite 47
Beispiel 5.16: aufraeumen.py … Seite 48
Beispiel 5.17: Test im Wegwerf-Ordner statt in den echten Downloads … Seite 49
Beispiel 5.18: ~/.config/systemd/user/aufraeumen.service (Trockenlauf-Fassung) … Seite 50
Beispiel 5.19: ~/.config/systemd/user/aufraeumen.timer … Seite 50
Beispiel 5.20: Timer einrichten, OHNE ihn sofort zu starten … Seite 50
Beispiel 5.20b: Erst nach kontrollierter Sichtung auf echten Betrieb umstellen … Seite 50
Beispiel 5.20c: Timer deaktivieren … Seite 51
Beispiel 6.1: Key aus der Umgebung lesen, nie im Code speichern … Seite 52
Beispiel 6.2: Nach Geheimnissen im Projekt suchen … Seite 53
Beispiel 6.3: Trockenläufe und sichere Gewohnheiten … Seite 54
Beispiel 6.4: Eigenen Testbenutzer anlegen (Debian/Ubuntu) … Seite 54
Beispiel 6.5: Unsicher und sicher – Befehle aus Eingaben bauen … Seite 54
Beispiel 6.5b: Erwartetes Ergebnis beider Varianten (Testeingabe: x; echo GEHACKT) … Seite 55
Beispiel 6.6: Pfade absichern … Seite 55
Beispiel 6.7: Parametrisierte SQL-Abfrage mit sqlite3 … Seite 55
Beispiel 6.8: Prüfroutine als kleines Skript … Seite 56
Beispiel 6.9: Ein Paket vor der Installation prüfen … Seite 56
Beispiel 6.10: Testdaten für eine Beispiel-CSV selbst erzeugen (keine echten Daten nötig) … Seite 58
Beispiel 7.1: Behauptungen schnell nachprüfen … Seite 60
Beispiel 7.2: Versionen feststellen … Seite 61
Beispiel 7.3: Versionsbewusster Prompt … Seite 61
Beispiel 7.4: Die richtige Umgebung prüfen … Seite 62
Beispiel 7.5: Virtuelle Umgebung sauber neu aufbauen … Seite 62
Beispiel 7.6: Prompt für Verständnis in Stufen … Seite 63
Beispiel 7.7: Verdächtige Fehlerbehandlung aufspüren … Seite 64
Beispiel 7.8: Zeilenenden und Zeichen prüfen und reparieren … Seite 64
Beispiel 8.1: Wochenplan für Stufe 2 … Seite 67
Beispiel 8.2: Der KI-Tutor-Prompt … Seite 67
Beispiel 8.3: Checkliste für jedes Projekt … Seite 70

Index

.gitignore – 19, 62
API-Key – 17, 18, 52
Branch – 67
chmod – 9, 18, 56
Code-Review – 34, 69
Command Injection – 54
Commit – 18, 19, 23, 24, 26, 37, 41, 53
Datenschutz – 57, 58, 68
Debugging – 27
Docker – 54
Exit-Code – 25, 26
Flask – 45, 46, 55, 68
Git – 6, 9, 10, 13, 14, 17, 18, 19
git diff – 26, 56, 63, 64
git restore – 29, 63
Halluzination – 9, 60
Hash – 37
jq – 42
Kontextfenster – 28
Lerntagebuch – 15, 29, 64
Linter – 35, 56
Lizenz – 57
localStorage – 42
Neovim – 16
Path Traversal – 55
PROJEKT.md – 24
Prompt – 23, 24, 31, 34, 35, 37, 38, 43
requirements.txt – 56
SHA-256 – 37
shellcheck – 35, 56
Slopsquatting – 56
SQL – 55
Standardbibliothek – 8, 28, 35, 56, 60, 63
systemd – 49, 50
Terminal-Agent – 9, 17, 24, 53
Traceback – 33
Trockenlauf – 46, 49, 50
Umgebungsvariable – 18, 52
unittest – 35, 56
venv – 15, 62, 64
Vibe Coding – 5, 8, 18, 23, 52, 62, 69, 70
VS Code – 16
XSS – 45, 55

VIBE CODING UNTER LINUX

Eigene Software bauen – mit KI und deinem Terminal

Du kennst die Kommandozeile, hast aber noch nie programmiert? Dieses Handbuch zeigt dir, wie du mit KI-Assistenten eigene Werkzeuge baust: vom ersten Prompt bis zum getesteten Programm. Du richtest deine Linux-Umgebung ein, arbeitest in kleinen, sicheren Schritten mit Git und lernst, KI-Code zu prüfen, statt ihm blind zu vertrauen.

DAS ERWARTET DICH

- Setup unter Linux: VS Code oder Neovim, Python, venv, Git, API-Keys
- Der Workflow: Prompt, Test, Commit – inklusive Debugging und Kontext-Management
- Drei Projekte Schritt für Schritt: CLI-Tool, Web-Projekt, Automatisierungsskript
- Sicherheit und Qualität: Keys, Lizenzen, Datenschutz, Grenzen von KI-Code
- Übungen, Warn- und Tipp-Boxen, Glossar, Verzeichnis der Beispiele, Index

Für wen?

Für Einsteiger:innen mit Linux-Grundkenntnissen. Keine Vorkenntnisse im Programmieren nötig. Praxisnah, Schritt für Schritt, mit Übungen.

76 Seiten · Stand Oktober 2026 · Für Einsteiger:innen mit Linux-Grundkenntnissen

Hinweis: Keine Rechts- oder Anlageberatung. Marken- und Produktnamen gehören ihren jeweiligen Inhabern.